

EBIS Business Techschool

Máster en Ingeniería y Desarrollo de Soluciones de IA Generativa

Seshat: Extracción y Estructuración del Conocimiento mediante Agentes LLM

Autor: Josep Martinez

Autor: Nombre Apellidos

Resumen

Las reuniones técnicas generan decisiones, riesgos y compromisos que con frecuencia quedan dispersos en notas informales o en la memoria de los participantes. Este trabajo presenta **Seshat**¹, un sistema que ingiere grabaciones de reuniones, extrae entidades estructuradas mediante agentes LLM y las persiste en una base de conocimiento con forma de grafo. Las relaciones entre decisiones a través de reuniones se rastrean explícitamente, permitiendo consultar la evolución del conocimiento organizativo a lo largo del tiempo.

Palabras clave: grandes modelos de lenguaje, extracción de información, grafos de conocimiento, recuperación aumentada por generación, búsqueda vectorial, orquestación de agentes, gestión del conocimiento organizativo.

¹Seshat (egipcio antiguo *ss3t*, lit. “*escriba femenina*”) era la diosa egipcia de la escritura, el conocimiento y la sabiduría. Considerada inventora de la escritura y guardiana de los registros, también presidía la arquitectura, la astronomía y la historia [1]. El nombre fue elegido por su resonancia con la misión del sistema: extraer, preservar y conectar el conocimiento generado en las reuniones.

Índice general

Resumen	ii
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Objetivos	2
1.2.1. Objetivo general	2
1.2.2. Objetivos específicos	2
1.3. Alcance de la solución	2
2. Estado del Arte y Fundamentos Teóricos	4
2.1. Grandes modelos de lenguaje	4
2.1.1. Arquitectura Transformer	4
2.1.2. Modelos de generación de texto	5
2.1.3. <i>Fine-tuning</i> y adaptación mediante prompts	5
2.2. Recuperación aumentada por generación	6
2.3. Procesamiento del habla	6
2.3.1. Reconocimiento automático del habla	6
2.3.2. Síntesis de voz	7
2.4. Consideraciones éticas y legales	7
3. Requisitos y Diseño de la Solución	8
3.1. Requisitos funcionales y no funcionales	8
3.1.1. Requisitos funcionales	8
3.1.2. Requisitos no funcionales	9
3.2. Arquitectura de la aplicación	9
3.3. Selección tecnológica	10
3.3.1. Lenguaje y frameworks	10
3.3.2. Proveedores de modelos externos	10
3.3.3. Almacenamiento	11
3.4. Diseño de datos y pipelines	11
3.4.1. Modelo de datos: el grafo de conocimiento	11
3.4.2. Ciclo de vida de un trabajo	12
4. Implementación	14
4.1. Estructura del código y componentes	14
4.2. Configuración centralizada	15
4.3. Ingesta	15
4.3.1. Validación	15
4.3.2. Transcripción	15
4.4. Extracción multi-agente en dos pasadas	16
4.4.1. Identificación	16
4.4.2. Puntuación de confianza y estado de aprobación	17
4.4.3. Recuperación aumentada	17

4.4.4.	Agentes de resolución	19
4.5.	Orquestación y ciclo de vida del pipeline	20
4.5.1.	API y creación del pipeline	20
4.5.2.	Worker asíncrono y máquina de estados	20
4.5.3.	Revisión humana (HITL)	21
4.6.	Mecanismos de seguridad	21
4.6.1.	Control de acceso y autenticación	21
4.6.2.	Validación de entradas	22
4.7.	Observabilidad y trazabilidad	22
4.7.1.	Trazado de la ejecución	22
4.7.2.	Medición del consumo y presupuesto	22
4.7.3.	Trazabilidad del conocimiento	23
4.8.	Despliegue e infraestructura	23
4.8.1.	Contenedores Docker	23
4.8.2.	Integración continua	23
5.	Evaluación del Sistema	24
5.1.	Estrategia de evaluación	24
5.2.	Corpus sintético de evaluación	25
5.3.	Métricas y resultados por arnés	27
5.3.1.	Identificación	27
5.3.2.	Agrupación	28
5.3.3.	Grounding	28
5.3.4.	Recuperación (retrieval)	29
5.3.5.	Resolución de relaciones	29
5.4.	Calibración de umbrales	31
5.4.1.	Umbral de confianza (identificación)	31
5.4.2.	Umbral de similitud (retrieval)	32
5.5.	Análisis de errores y limitaciones	32
6.	Pruebas y Validación de Producto	33
6.1.	Tests unitarios y de integración	33
6.1.1.	Pruebas unitarias	33
6.1.2.	Pruebas de integración	33
6.2.	Eval gate y regresión automatizada	34
6.3.	Experiencia de usuario (UX)	35
7.	Discusión	36
7.1.	Lecciones aprendidas	36
7.2.	Riesgos, ética y mitigaciones	36
7.2.1.	Alucinaciones y fiabilidad de los agentes	37
7.2.2.	Privacidad de las conversaciones	37
7.2.3.	Dependencia de proveedores externos	37
8.	Conclusiones y Trabajo Futuro	38
8.1.	Conclusiones	38
8.2.	Trabajo futuro	39
8.2.1.	Endurecimiento para producción	39
8.2.2.	Extensiones del núcleo generativo	39
8.2.3.	Profundización del grafo de conocimiento	40

8.2.4. Integración y nuevas fuentes de entrada	41
A. Guía de Despliegue y Manual de Usuario	42
A.1. Requisitos previos	42
A.2. Configuración de credenciales	42
A.3. Arranque con Docker Compose	43
A.4. Manual de usuario	44
B. Detalles Técnicos	45
B.1. Arquitectura de agentes	45
B.1.1. Jerarquía de clases y registro	45
B.1.2. Agentes reflectivos	46
B.1.3. Matriz de relaciones por par de tipos	47
B.1.4. Técnicas de diseño de prompts	47
B.1.5. Caché de prompts y optimización de latencia	48
B.2. Capa de observabilidad	49
B.3. Arquitectura de evaluación	49
B.3.1. Estructura común de los arneses y el emparejador de identificación .	49
B.3.2. Caché de evaluación y meta-scorers de calibración	50
B.4. Esquema de la base de datos	51
B.5. Referencia de la API REST	53
B.6. Configuración completa de referencia	54
B.7. <i>Prompts</i> de los agentes	54
Acrónimos	58
Bibliografía	59

1. Introducción

1.1. Contexto y motivación

Los equipos de ingeniería toman decisiones continuamente, tales como qué arquitectura adoptar, qué riesgos aceptar, qué tareas asignar y qué preguntas quedan abiertas. Una parte importante de ese proceso ocurre en reuniones técnicas, donde la conversación directa facilita la deliberación y la toma de acuerdos. Sin embargo, el conocimiento generado en esas reuniones rara vez queda registrado de manera estructurada: termina disperso en notas informales, hilos de mensajería instantánea y en la memoria de los participantes, sin trazabilidad del razonamiento que condujo a cada decisión.

Esta brecha entre el conocimiento producido y el conocimiento preservado tiene consecuencias prácticas: cuando un nuevo miembro del equipo necesita entender por qué se eligió una base de datos en particular, o cuando un riesgo identificado meses atrás resurge en un contexto diferente, no existe un registro consultable que responda a esas preguntas. La ausencia de un grafo de decisiones explícito obliga a reconstruir el contexto a partir de fuentes fragmentadas, con el coste de tiempo y el riesgo de inconsistencia que ello conlleva.

Existen herramientas que transcriben y resumen reuniones automáticamente, pero su salida es texto no estructurado: un resumen o una transcripción, no un conjunto de entidades consultables. Ninguna de ellas distingue entre una decisión, un riesgo o una tarea; ninguna rastrea cómo una decisión tomada en marzo contradice o matiza otra tomada en junio. La estructuración del conocimiento, su tipado y la trazabilidad de su evolución a lo largo del tiempo quedan fuera de su alcance.

Los Large Language Model (LLM) han transformado las capacidades de procesamiento del lenguaje natural. Desde la introducción de la arquitectura Transformer [2], los modelos de lenguaje han escalado en capacidad hasta alcanzar rendimiento notable en tareas de comprensión y generación de texto [3]. Esta evolución ha abierto la puerta a sistemas capaces de extraer información estructurada a partir de texto no estructurado de forma fiable, sin necesidad de entrenamiento supervisado específico por dominio.

Al mismo tiempo, la técnica de Retrieval-Augmented Generation (RAG) [4] ha demostrado que combinar la búsqueda en bases de conocimiento externas con la generación de modelos de lenguaje mejora significativamente la precisión y trazabilidad de las respuestas. Esta combinación resulta especialmente relevante en contextos donde el conocimiento evoluciona a lo largo del tiempo y la coherencia entre distintas fuentes es crítica.

El presente Trabajo de Fin de Máster (TFM) presenta **Seshat**: un sistema que ingiere grabaciones de reuniones técnicas, extrae entidades estructuradas mediante una cadena de agentes LLM, y las persiste en una base de conocimiento con forma de grafo. El diseño incorpora un paso de revisión humana, asistida por puntuación de confianza; un arnés de evaluación reproducible que permite comparar el rendimiento del sistema cuando se introducen cambios, y una capa de observabilidad basada en MLflow. El resultado es un sistema completo, desplegable mediante Docker Compose, orientado a equipos de ingeniería que necesitan documentar y consultar el conocimiento técnico generado en sus reuniones.

1.2. Objetivos

1.2.1. Objetivo general

El objetivo principal del proyecto es diseñar e implementar un sistema de extracción automática de conocimiento a partir de grabaciones de reuniones técnicas, capaz de identificar elementos clave de forma estructurada, gestionar las relaciones entre ellas a lo largo del tiempo mediante un grafo de conocimiento, e incorporar revisión humana asistida y evaluación calibrada como requisitos de calidad.

1.2.2. Objetivos específicos

Por simplicidad, se descompone el objetivo general en los siguientes objetivos específicos:

1. Diseñar un flujo multi-agente que extraiga entidades estructuradas de cuatro tipos a partir de transcripciones de reuniones e infiera las relaciones semánticas entre ellas y el conocimiento preexistente.
2. Incorporar revisión humana asistida por puntuación de confianza, de modo que el revisor pueda concentrar su atención en los nodos de menor certeza y el sistema pueda operar en modo automático cuando la confianza lo permita.
3. Construir un marco de evaluación con corpus etiquetado que mida el rendimiento mediante de forma reproducible, usando métricas como *precision*, *recall* o derivadas de ellas y bloquee el procesamiento de datos reales si no se alcanza la calidad definida.
4. Desarrollar una Application Programming Interface (API) REST y una interfaz web que cubran el ciclo de vida completo del trabajo (envío, transcripción, extracción, revisión y consulta del grafo) sin requerir acceso directo a la capa de persistencia.
5. Desplegar el sistema como servicios contenerizados con observabilidad integrada, de modo que el rendimiento y el coste sean medibles y comparables entre versiones.

1.3. Alcance de la solución

El sistema admite dos tipos de entrada: archivos de audio (.mp3, .wav, .m4a) transcritos mediante un servicio externo, y transcripciones preformateadas (.yaml o .json). Ambas convergen en una representación interna común antes de entrar en el proceso de extracción.

La extracción cubre cuatro tipos de nodo y siete tipos de relación semántica entre ellos, detallados en el capítulo 3. La base de conocimiento sigue un modelo de escritura por estado: el contenido de un nodo aprobado es estable por diseño, aunque operadores y administradores pueden corregirlo mediante un mecanismo de anulación explícita que registra el motivo y el autor del cambio. El único campo que cambia de forma automática es el estado del nodo, que en esta primera versión se actualiza cuando una reunión posterior establece una relación de supersesión o enmienda.

El sistema incluye una interfaz gráfica para la revisión humana de los nodos extraídos, un mecanismo de aprobación masiva por umbral de confianza y la posibilidad de que el operador edite nodos manualmente durante la revisión. Las relaciones inferidas se escriben automáticamente tras la aprobación de los nodos. La observabilidad está cubierta mediante MLflow [5]: cada ejecución registra el uso de tokens, la latencia por etapa y los resultados de las métricas de evaluación.

El sistema distingue tres tipos de trabajo: el *pipeline* de ingesta y extracción, que constituye el flujo principal descrito en este documento; los trabajos de inicialización¹, que preparan la base de conocimiento antes de que el sistema entre en operación; y las operaciones manuales del operador, como la adición directa de nodos durante la revisión. En lo que sigue, el término *pipeline* se reserva para el flujo de ingesta y extracción.

Esta versión inicial (MVP) deja fuera de su alcance un conjunto de capacidades que se agrupan en cuatro categorías según la razón de su aplazamiento. La primera es el **endurecimiento para producción**: el despliegue en infraestructura cloud, con su definición mediante Infrastructure as Code (IaC), y la autenticación con un proveedor de identidad externo (*SSO*) exceden lo que un prototipo de investigación necesita demostrar. La segunda son las **extensiones del núcleo generativo**, funcionalidades afines a los objetivos pero pospuestas por tiempo o por depender de datos de producción: la detección y anonimización de información personal identificable (PII), la recuperación agéntica como alternativa al *pipeline* RAG [6], o una señal de confianza complementaria basada en inferencia de lenguaje natural² (NLI). La tercera es la **profundización del grafo de conocimiento**, trabajo de modelado de datos más que de Inteligencia Artificial Generativa (IAG): la revisión humana de las relaciones inferidas o el uso de un *backend* de grafo nativo como Neo4j. La cuarta abarca la **integración y nuevas fuentes de entrada**: la entrada de vídeo, la diarización³ de locutores, y la inicialización de la base desde un corpus documental existente —por ejemplo desde [Notion](#) o [Confluence](#)—. El capítulo 8 retoma y detalla estas líneas como trabajo futuro (sección 8.2).

¹Los trabajos de inicialización, *init*, poblarían la base de conocimiento a partir de una fuente distinta de una reunión —por ejemplo, un corpus documental existente— antes de la puesta en marcha del sistema. Su implementación se aplaza como trabajo futuro (capítulo 8, sección 8.2).

²La *inferencia de lenguaje natural* (Natural Language Inference (NLI)) es la tarea de determinar si una hipótesis se puede deducir (*entailment*), contradice (*contradiction*) o es neutral respecto a una premisa dada. Aplicada a la puntuación de confianza, un modelo NLI verifica si la descripción del nodo extraído está respaldada por su cita fuente, aún usando sinónimos. Es, por ahora, una propuesta analizada pero no implementada: no se ha verificado si aportaría una señal realmente independiente de las heurísticas ya existentes.

³La *diarización* (*speaker diarization*) es el proceso de segmentar una grabación de audio en intervalos etiquetados por locutor, respondiendo a la pregunta «¿quién habló cuándo?».

2. Estado del Arte y Fundamentos Teóricos

2.1. Grandes modelos de lenguaje

2.1.1. Arquitectura Transformer

Hasta 2017, los modelos dominantes en procesamiento del lenguaje natural eran las redes recurrentes, que procesan el texto token a token acumulando el contexto en un vector de estado oculto. No obstante, esa cadena de multiplicaciones hace que los gradientes se desvanezcan o exploten durante el entrenamiento, dificultando el aprendizaje de dependencias entre palabras lejanas¹. La propuesta de Vaswani et al. [2] fue eliminar la recurrencia por completo: en lugar de propagar el contexto de forma secuencial, cada token puede relacionarse directamente con cualquier otro de la secuencia en un único paso paralelizable.

El mecanismo que lo hace posible es la **atención**. A partir de la entrada se derivan, mediante proyecciones lineales de la matriz de embeddings, tres matrices: Q (*queries*), que representa qué información busca cada posición; K (*keys*), que representa qué información ofrece cada posición para ser encontrada; y V (*values*), que contiene el contenido que se recupera cuando una posición es seleccionada. La atención se calcula como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (2.1)$$

El producto $QK^\top$ mide la afinidad entre cada par de posiciones; dividirlo por $\sqrt{d_k}$ evita que los valores crezcan tanto que el *softmax* sature y los gradientes desaparezcan. El resultado es una suma ponderada de los valores V , donde los pesos reflejan cuánto «presta atención» cada posición a las demás.

Una sola aplicación de esta operación captura un tipo de relación a la vez. Por eso la arquitectura usa *multi-head attention*: aplica la ecuación (2.1) en paralelo en múltiples subespacios proyectados independientemente, cada uno libre de especializarse en un patrón distinto —sintáctico, semántico, correferencial— y concatena las salidas al final. Así, el modelo puede razonar simultáneamente sobre estructura local y dependencias globales.

La arquitectura original combina dos bloques con roles complementarios. El **codificador** (*encoder*) procesa la secuencia de entrada completa con atención bidireccional, produciendo una representación contextualizada de cada token; es la elección natural para tareas de comprensión. El **decodificador** (*decoder*) genera la salida token a token de forma autoregresiva, usando atención enmascarada para no ver posiciones futuras y atención cruzada sobre la salida del codificador para anclar la generación en la entrada.

¹El problema del gradiente desvaneciente fue estudiado por Hochreiter (1991) y formalizado por Bengio et al. (1994). LSTM y GRU paliaron el síntoma introduciendo compuertas, pero no eliminaron la causalidad secuencial.

Ambos bloques inyectan codificaciones posicionales sinusoidales para preservar el orden sin recurrir a la recurrencia. Escalada en datos y parámetros, esta arquitectura dio lugar a los grandes modelos de lenguaje actuales.

2.1.2. Modelos de generación de texto

Los grandes modelos de lenguaje para generación de texto adoptan una variante *decoder-only* del Transformer: eliminan el bloque codificador y emplean únicamente el decodificador con atención causal sobre toda la secuencia. Esta simplificación concentra el entrenamiento en un único objetivo —predecir el siguiente token sobre el corpus completo— y ha demostrado escalar de forma especialmente eficiente a cientos de miles de millones de parámetros. Modelos como GPT-5, Claude o Gemini responden a esta arquitectura.

Un hallazgo determinante sobre estos modelos llegó con Brown et al. [3]: los modelos con suficiente capacidad exhiben *few-shot learning*, es decir, dados unos pocos ejemplos en el propio prompt son capaces de generalizar la tarea sin actualizar sus pesos. Esto redujo drásticamente la dependencia de conjuntos de datos etiquetados para cada aplicación específica y abrió la puerta a sistemas adaptables sin *fine-tuning*.

El siguiente paso fue el *fine-tuning*. Los modelos preentrenados son competentes, pero no necesariamente útiles: pueden generar texto coherente sin seguir instrucciones ni respetar preferencias humanas. El aprendizaje por refuerzo a partir de retroalimentación humana (*RLHF*) resolvió esto en dos fases: primero, evaluadores humanos clasifican pares de respuestas del modelo, entrenando un modelo de recompensa que aprende a predecir qué respuesta preferiría un humano; después, el modelo de lenguaje se ajusta mediante aprendizaje por refuerzo para maximizar esa recompensa. El resultado es un modelo que sigue instrucciones, reconoce cuando no sabe algo y produce salidas estructuradas con mucha mayor fiabilidad. Son estos modelos — preentrenados a escala y alineados con instrucciones— los que **Seshat** emplea como base de sus agentes.

2.1.3. *Fine-tuning* y adaptación mediante prompts

El *fine-tuning* permite especializar un modelo preentrenado en un dominio concreto mediante entrenamiento adicional sobre datos etiquetados. Aunque eficaz, introduce costes de datos, cómputo y mantenimiento que lo hacen poco adecuado para sistemas que deben adaptarse a cambios frecuentes de modelo base o de dominio.

Una alternativa más ligera es el *prompt engineering*, que explota la capacidad de los LLM de generalizar a partir del contexto sin modificar sus pesos. Más allá de proporcionar ejemplos de salida esperada (*few-shot learning*), el diseño cuidadoso de prompts para tareas de extracción estructurada combina varias técnicas complementarias: el razonamiento en cadena [7] descompone la tarea en pasos intermedios verificables antes de emitir la respuesta; el anclaje en cita literal obliga al modelo a localizar el fragmento de texto que respalda cada nodo antes de describirlo, reduciendo la alucinación; las compuertas lógicas codifican condiciones de inclusión y exclusión que el modelo debe evaluar explícitamente; y los contraejemplos etiquetados delimitan fronteras semánticas entre categorías próximas. La salida estructurada forzada mediante esquemas Pydantic elimina la ambigüedad de formato y habilita reintentos automáticos ante violaciones de tipo. **Seshat** opta por este segundo enfoque; las técnicas concretas se describen en el anexo B.1.4.

2.2. Recuperación aumentada por generación

Los LLM tienen un límite estructural: su conocimiento queda congelado en el momento del entrenamiento. La generación aumentada por recuperación (RAG) [4] aborda este problema combinando el modelo generativo con un módulo de recuperación: antes de generar, el sistema consulta una base de conocimiento externa, selecciona los fragmentos más relevantes e incluye estos en el prompt. La respuesta queda así anclada en evidencia concreta y el conocimiento del sistema puede actualizarse sin re-entrenar el modelo.

La pieza central del módulo de recuperación son los *embeddings*: representaciones vectoriales densas que codifican el significado semántico de un texto en un espacio de alta dimensión, de modo que textos con significado similar queden cerca en ese espacio. La semejanza entre vectores, medida típicamente mediante similitud coseno, permite identificar los fragmentos más relevantes para una consulta dada. Bases de datos vectoriales como **pgvector** proporcionan la infraestructura de indexación y búsqueda eficiente sobre estas representaciones.

El paradigma ha evolucionado más allá de la búsqueda plana. Para mejorar la cobertura, se han propuesto estrategias de expansión de consulta: en lugar de buscar con la pregunta original, el sistema genera múltiples variantes con un LLM y fusiona los resultados, diversificando los candidatos recuperados [8]. En la etapa de selección, el reranking con modelos *cross-encoder* [9] mejora la precisión al comparar directamente la consulta con cada candidato, en lugar de aproximar la relevancia con la similitud coseno. GraphRAG [10] va más allá y construye un grafo de entidades a partir de un corpus documental para enriquecer la recuperación con contexto estructural. **Seshat** incorpora las dos primeras estrategias—multi-query y reranking opcional— e invierte la premisa de GraphRAG: usa RAG no para responder preguntas, sino para construir y mantener el propio grafo, recuperando los nodos preexistentes más relevantes para determinar qué relaciones semánticas introduce cada nuevo nodo extraído.

2.3. Procesamiento del habla

2.3.1. Reconocimiento automático del habla

Hasta mediados de la década pasada, los sistemas de reconocimiento automático del habla (Automatic Speech Recognition (ASR)) se construían como cadenas de módulos especializados: un modelo acústico que asignaba probabilidades a fonemas, un modelo de lenguaje que los combinaba en palabras y un decodificador que buscaba la secuencia más probable. El aprendizaje profundo fue reemplazando estos módulos por redes neuronales entrenadas conjuntamente, y la arquitectura Transformer completó la transición hacia modelos *encoder-decoder* que aprenden directamente la correspondencia entre audio y texto sin alineamiento explícito.

Whisper [11] demostró hasta dónde podía llegar este enfoque: entrenado sobre 680.000 horas de audio con transcripciones débilmente supervisadas obtenidas de internet, produce un modelo robusto y multilingüe que no requiere *fine-tuning* para nuevos dominios. Su codificador procesa espectrogramas de log-Mel y su decodificador genera la transcripción de forma autoregresiva.

Seshat delega la transcripción en AssemblyAI, un servicio de la misma familia. La interfaz de transcripción es abstracta y sustituible sin modificar la lógica del *pipeline*.

2.3.2. Síntesis de voz

La síntesis de voz (*Text-to-Speech*, TTS) siguió un arco similar. Los sistemas clásicos producían voz inteligible pero perceptiblemente artificial. WaveNet [12] fue el primer modelo neuronal capaz de generar formas de onda con naturalidad comparable a la voz humana, aunque con un coste computacional prohibitivo para tiempo real. Los modelos posteriores separaron la generación del espectrograma de mel² de la síntesis de la forma de onda, reduciendo la latencia. VITS [13] unificó ambas etapas: combina un autocodificador variacional con un modelo de flujo normalizado y entrenamiento adversarial para generar audio de alta calidad en una sola pasada.

En este trabajo, la TTS no forma parte del *pipeline* en tiempo de ejecución. ElevenLabs, un servicio comercial de alta fidelidad, se empleó exclusivamente fuera de línea para generar grabaciones sintéticas de reuniones multi-locutor —con ruido de fondo de oficina opcional— que ejercitan la ruta de transcripción e ingesta de audio de extremo a extremo.

2.4. Consideraciones éticas y legales

Las transcripciones de reuniones técnicas pueden contener información sensible: datos personales, decisiones estratégicas confidenciales o referencias a terceros. El Reglamento General de Protección de Datos (RGPD) exige que el tratamiento de datos personales cuente con una base legal, que los interesados sean informados y que los datos se almacenen de forma segura. **Seshat** no implementa en su versión actual ningún mecanismo de detección ni anonimización de información personal identificable (PII); esta capacidad queda identificada como línea de trabajo futuro y se aborda en el capítulo 7.

Un aspecto igualmente relevante es el consentimiento informado de los participantes: más allá del cumplimiento del RGPD, la existencia de un sistema que estructura permanentemente el contenido de las reuniones puede desincentivar la comunicación abierta si los participantes no son conscientes de su funcionamiento. Para uso interno, esto es una cuestión de confianza organizativa tanto como de obligación legal.

Más allá de la privacidad y el consentimiento, los LLM introducen un riesgo propio: las alucinaciones, es decir, la generación de afirmaciones falsas pero plausibles. Estas pueden dar lugar a nodos incorrectos en la base de conocimiento, con consecuencias reales si esos nodos informan decisiones futuras. **Seshat** mitiga este riesgo mediante la revisión humana de los nodos de baja confianza, el anclaje de cada nodo a una cita literal de la transcripción y el registro completo de prompts e inferencias en MLflow [5] para auditoría.

Un tercer vector de riesgo es la inyección de prompt: un actor que controle el contenido de la transcripción podría insertar instrucciones que manipulen el comportamiento del agente. **Seshat** lo previene mediante delimitadores explícitos que separan el contenido procesado de las instrucciones del sistema, impidiendo que el texto de entrada pueda reinterpretar el rol del agente.

²Un espectrograma de mel representa cómo varía en el tiempo la energía de una señal de audio en distintas bandas de frecuencia, con una escala (*mel*) que espacia esas bandas de forma similar a como el oído humano percibe el tono, en lugar de linealmente en Hz.

3. Requisitos y Diseño de la Solución

3.1. Requisitos funcionales y no funcionales

3.1.1. Requisitos funcionales

Los requisitos siguientes describen el comportamiento esperado del sistema desde la perspectiva del usuario y del operador.

1. **Interfaz de acceso.** El sistema debe exponer todas sus funcionalidades a través de una API REST como único punto de entrada programático. Las operaciones de ingesta, consulta y revisión deben ser accesibles sin intervención de la interfaz gráfica.
2. **Ingesta de reuniones.** El sistema debe aceptar grabaciones de audio en formato MP3, WAV o M4A, así como transcripciones preformateadas en YAML o JSON. Ambas rutas deben converger en una representación interna común antes de continuar el procesamiento.
3. **Extracción estructurada.** A partir de la transcripción, el sistema debe identificar nodos de cuatro tipos: decisiones (`decision`), riesgos (`risk`), preguntas abiertas (`open_question`) e ítems de acción (`action_item`). Cada nodo debe quedar anclado a una cita literal del texto fuente.
4. **Resolución de relaciones.** El sistema debe inferir relaciones semánticas entre los nodos extraídos y los nodos preexistentes en la base de conocimiento, clasificándolas en siete tipos: `supersedes`, `amends`, `conflicts_with`, `depends_on`, `mitigates`, `blocks` y `resolves`.
5. **Puntuación de confianza.** Cada nodo extraído debe recibir una puntuación de confianza en $[0, 1]$ antes de la revisión humana. La puntuación combina señales heurísticas basadas en reglas y, opcionalmente, un agente de verificación independiente.
6. **Revisión y corrección humana.** Los nodos con puntuación inferior al umbral configurado deben quedar en estado de revisión pendiente. Un revisor humano debe poder aprobar, rechazar o editar cada nodo antes de que éste entre en la base de conocimiento. Debe existir también un mecanismo de aprobación masiva por umbral.
7. **Base de conocimiento consultable.** Los nodos aprobados deben persistir en una base de conocimiento con forma de grafo, consultable por tipo, estado, equipo, proyecto, dominio y rango de fechas. El sistema debe permitir búsqueda semántica y travesía del grafo para explorar el impacto de un nodo sobre otros.
8. **Marco de evaluación.** El sistema debe incluir un marco de evaluación funcionalmente determinista con corpus etiquetado, que mida la calidad de extracción y bloquee el procesamiento de datos reales si no se alcanzan los objetivos de calidad definidos.

3.1.2. Requisitos no funcionales

Los requisitos siguientes describen las propiedades de calidad que el sistema debe satisfacer con independencia de la funcionalidad concreta que se esté ejecutando.

- **Trazabilidad.** Cada nodo debe referenciar la cita literal del texto fuente que lo originó. Todos los prompts e inferencias deben quedar registrados para auditoría.
- **Seguridad.** El acceso a la API debe estar controlado mediante claves de API con modelo de roles. Las entradas de audio deben validarse por contenido (firma de bytes), no por extensión o cabecera. Las instrucciones del sistema deben estar aisladas del contenido procesado para prevenir la inyección de prompts.
- **Extensibilidad.** Los proveedores de LLM, transcripción, almacenamiento vectorial y secretos deben ser intercambiables sin modificar la lógica del *pipeline*. Los cambios de proveedor deben ser una modificación de configuración, no una refactorización.
- **Observabilidad.** El sistema debe registrar el uso de tokens, la latencia por etapa, las métricas de confianza y los errores en MLflow [5] de forma automática, sin instrumentación manual en los agentes.
- **Reproducibilidad.** El marco de evaluación debe producir resultados consistentes entre ejecuciones sobre el mismo corpus, dentro de los límites del no determinismo inherente a los LLM.
- **Atomicidad.** Cada escritura en la base de conocimiento debe ser una transacción única. No puede existir estado parcial tras un fallo.
- **Resiliencia.** Las llamadas a proveedores externos —LLM, transcripción, reranking, almacenamiento— deben estar sujetas a reintentos con *backoff* exponencial y *jitter*, y a tiempos de espera máximos configurables. El consumo de tokens debe estar acotado mediante límites por llamada y por ejecución, tanto para LLM como para embeddings. El sistema debe continuar operando ante fallos transitorios sin requerir intervención manual.

3.2. Arquitectura de la aplicación

Seshat se organiza en cuatro capas con flujo de control estrictamente unidireccional, tal como muestra la figura 3.1.

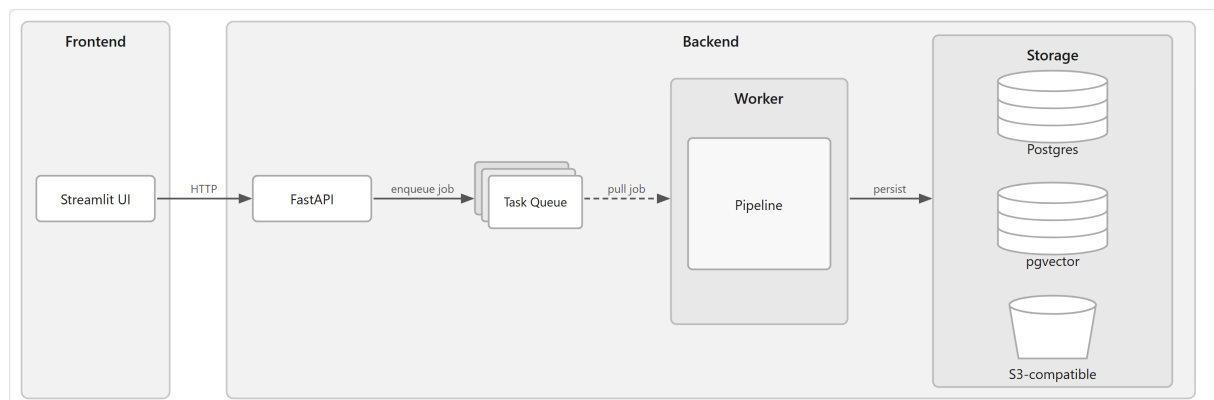


Figura 3.1.: Arquitectura de **Seshat**: la interfaz envía trabajos a la API por HTTP; la API los encola para el Worker; el Worker ejecuta el procesamiento y persiste los resultados en la capa de almacenamiento.

Cada capa tiene una responsabilidad delimitada, que se describe a continuación:

1. **Interfaz de usuario:** envía trabajos, muestra su estado y presenta los nodos pendientes de revisión al operador. Se comunica exclusivamente con la API por HTTP.
2. **API:** autentica y valida las peticiones, registra el trabajo en la base de datos y lo encola para su procesamiento asíncrono.
3. **Worker:** consume los trabajos de la cola y ejecuta el procesamiento correspondiente, persistiendo los resultados en la capa de almacenamiento.
4. **Capa de almacenamiento:** Postgres para el estado operativo y la base de conocimiento; pgvector para los embeddings; almacenamiento de objetos compatible con S3 para artefactos de audio, transcripciones y resultados de extracción.

3.3. Selección tecnológica

3.3.1. Lenguaje y frameworks

Seshat está implementado en Python, el lenguaje dominante en el ecosistema de IA generativa. Las principales dependencias son:

- **FastAPI:** API REST asíncrona con validación de esquemas y documentación OpenAPI autogenerada.
- **LangChain:** orquestación de agentes LLM con soporte para múltiples proveedores; integra pgvector como almacén vectorial mediante `langchain-postgres`.
- **MLflow:** observabilidad: registro automático de trazas, uso de tokens, latencia y métricas de evaluación.
- **Pydantic:** validación de modelos de datos y configuración centralizada desde variables de entorno.

3.3.2. Proveedores de modelos externos

Todos los proveedores son intercambiables mediante configuración. El sistema distingue cuatro roles:

- **Transcripción:** AssemblyAI por defecto; sustituible por OpenAI Whisper u otros mediante la interfaz `AbstractTranscriber`.
- **LLM:** OpenAI, Anthropic, Azure OpenAI y Amazon Bedrock Converse, todos a través de la abstracción de LangChain.
- **Embeddings:** OpenAI, Azure OpenAI y Cohere sobre pgvector, otra vez mediante la abstracción de LangChain.
- **Reranking:** Cohere y Voyage como paso opcional de reordenación de candidatos RAG, encapsulados detrás de la interfaz `AbstractReranker`.

Un requisito de diseño relevante afecta al agente de verificación de confianza (*grounding*): debe usar un proveedor distinto al de los agentes de identificación, ya que verificar con el mismo modelo produce errores correlacionados que anulan el valor de la señal [14], [15].

3.3.3. Almacenamiento

La capa de almacenamiento combina tres tecnologías con roles claramente separados:

- **PostgreSQL**: base de datos principal; almacena el estado operativo (trabajos, claves de API) y los nodos y relaciones de la base de conocimiento.
- **pgvector**: extensión de Postgres para búsqueda por similitud vectorial. Elimina la necesidad de un servicio vectorial separado y mantiene las escrituras en la base de conocimiento y los embeddings sincronizadas.
- **S3 / LocalStack**: almacenamiento de artefactos: audio original, transcripciones y resultados de extracción. LocalStack emula la API de AWS en desarrollo.

La gestión de secretos (claves de API, cadena de conexión de base de datos, etc) se delega en AWS Secrets Manager, también emulado por LocalStack en desarrollo.

3.4. Diseño de datos y pipelines

3.4.1. Modelo de datos: el grafo de conocimiento

El conocimiento extraído de las reuniones se representa como un grafo de conocimiento [16]: un grafo dirigido y etiquetado donde, los nodos pueden ser entidades de cuatro tipos (ver tabla 3.1) y las aristas son relaciones semánticas entre ellos de siete tipos, agrupados en dos categorías (ver tabla 3.2). La dimensión temporal es inherente al modelo: cuando una reunión posterior introduce un nodo que reemplaza o matiza a uno anterior, la relación queda registrada explícitamente y el estado del nodo destino se actualiza en consecuencia. El grafo preserva así la historia completa del conocimiento organizativo, no sólo su estado actual.

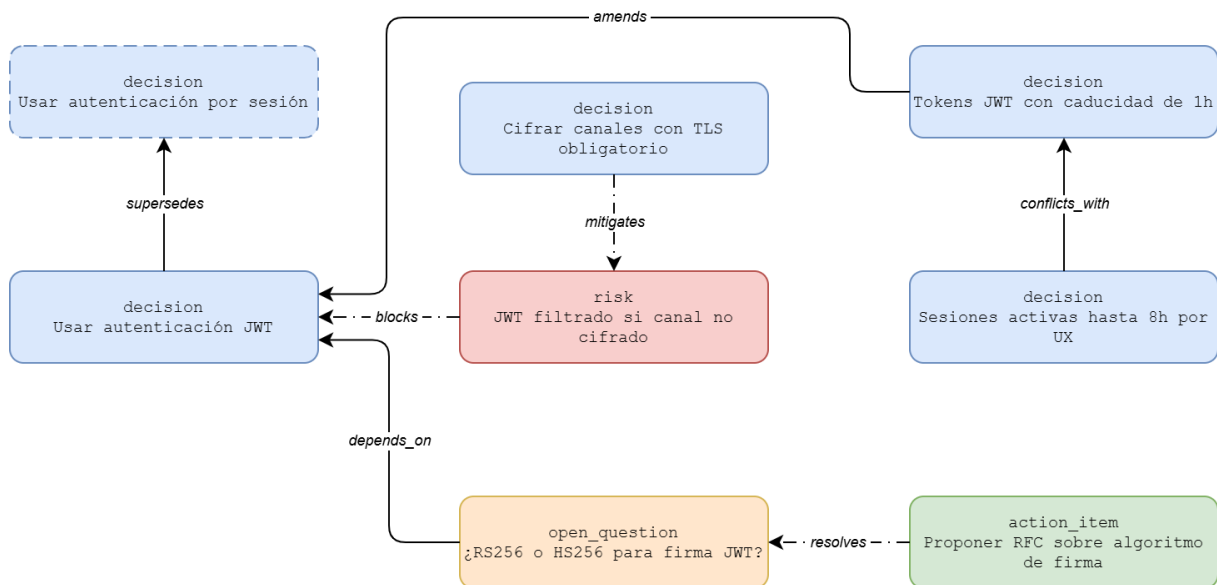


Figura 3.2.: Ejemplo de grafo de conocimiento en **Seshat**. El nodo con borde discontinuo representa una decisión previa en estado **SUPERSEDED**; las relaciones entre nodos de distinto tipo se muestran con flechas de línea discontinua.

Tipo	Descripción
<code>decision</code>	Compromiso explícito y de ámbito grupal sobre qué opción adoptar, qué política seguir o qué restricción imponer. Tiene consecuencias operativas más allá de la conversación.
<code>risk</code>	Modo de fallo concreto, bloqueante activo o incertidumbre con una consecuencia declarada que el grupo trata como sustantiva y no resuelta.
<code>open_question</code>	Pregunta cuya respuesta el grupo necesita pero aún no ha determinado; su resolución condiciona decisiones o trabajo futuro.
<code>action_item</code>	Tarea asignada a una persona o equipo con resultado esperado y, típicamente, fecha límite.

Tabla 3.1.: Tipos de nodo en el grafo de conocimiento de **Seshat**.

Tipo	Categoría	Definición
<code>supersedes</code>	Mismo tipo	El nodo fuente reemplaza permanentemente al destino
<code>amends</code>	Mismo tipo	El nodo fuente matiza o actualiza parcialmente al destino
<code>conflicts_with</code>	Mismo tipo	Los dos nodos están en contradicción activa
<code>depends_on</code>	Mismo tipo	El nodo fuente presupone al destino
<code>mitigates</code>	Tipo cruzado	Un riesgo queda mitigado por una decisión
<code>blocks</code>	Tipo cruzado	Un riesgo bloquea una decisión o pregunta abierta
<code>resolves</code>	Tipo cruzado	Una decisión o tarea cierra una pregunta abierta

Tabla 3.2.: Tipos de relación en el grafo de conocimiento de **Seshat**.

Cada nodo almacena su contenido —título, descripción y cita literal del texto fuente— junto con una puntuación de confianza y metadatos de auditoría. El contenido es estable por diseño una vez aprobado; el único campo que cambia automáticamente es el estado, actualizado cuando una reunión posterior introduce una relación de supersesión o enmienda.

El grafo en su conjunto sigue un principio de *append-and-state-only*: no existe operación de borrado ni sobrescritura. El único matiz es que el contenido de un nodo aprobado puede corregirse mediante una operación explícita reservada a operadores y administradores, que exige justificación escrita y queda registrada en el historial de auditoría.

3.4.2. Ciclo de vida de un trabajo

Un trabajo *pipeline* en **Seshat** representa el procesamiento completo de una reunión, desde la recepción del audio o texto hasta la persistencia de los resultados en la base de conocimiento. Recorre seis estados principales: **PENDING**, **TRANSCRIBING**, **IDENTIFYING**, **AWAITING_REVIEW**, **RESOLVING** y **WRITING**, más los estados terminales **DONE** y **FAILED**. Las dos fases de trabajo intensivo con LLM —identificación de nodos y resolución de relaciones— están separadas por la revisión humana opcional (*Human-in-the-Loop*, HITL): si hay nodos pendientes de aprobación y el modo automático no está activo, el *pipeline* se detiene en **AWAITING_REVIEW** hasta que el operador interviene. Los fallos recuperables permiten reintentar el trabajo; los fatales requieren un nuevo envío.

La figura 3.3 muestra las transiciones posibles.

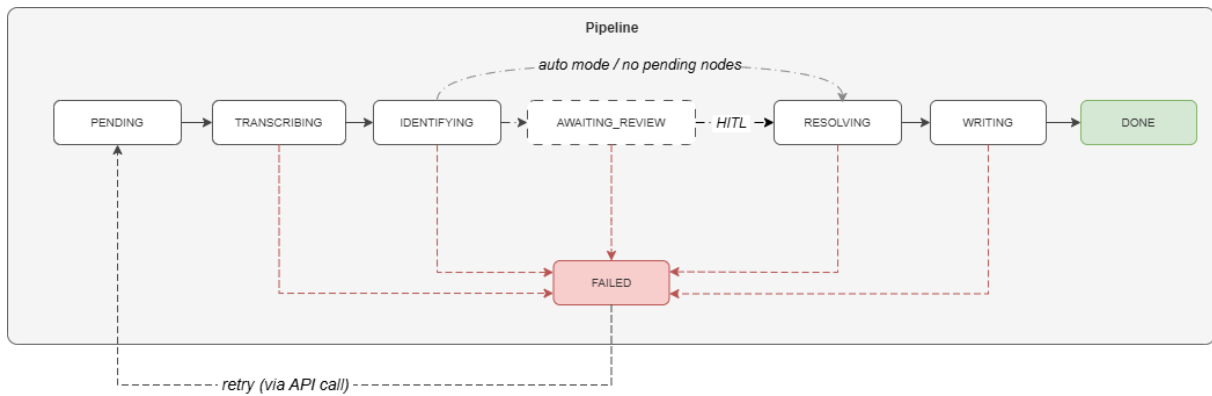


Figura 3.3.: Ciclo de vida de un *pipeline* en **Seshat**. IDENTIFYING y RESOLVING corresponden a las dos pasadas de extracción; el paso *HITL* indica la revisión humana opcional; y en WRITING se persisten los nodos y sus relaciones en la base de conocimiento. Las flechas discontinuas rojas señalan transiciones a error y la flecha inferior muestra el reintento mediante llamada a la API.

4. Implementación

4.1. Estructura del código y componentes

El paquete `seshat` se organiza en cinco módulos (figura 4.1): `core/` contiene los modelos, configuración y utilidades; `infra/` agrupa los adaptadores de sistemas externos; `app/` es la capa de aplicación; `eval/` contiene los arneses de evaluación, que no forman parte del sistema en producción; y `cli/` expone la línea de comandos.



Figura 4.1.: Estructura de paquetes de **Seshat**.

4.2. Configuración centralizada

Toda la configuración se centraliza en un único objeto `SeshatConfig`¹, que hereda de `pydantic_settings.BaseSettings`, con `__` como separador de anidamiento. Sus objetos anidados son subclases de `pydantic.BaseModel` planos de modo que la resolución de variables de entorno ocurre exclusivamente en la raíz. El objeto resultante es un *singleton* inmutable cargado en el arranque del proceso.

Para los trabajos que requieren parámetros diferentes a los globales —por ejemplo, un umbral de confianza más estricto o un modo automático activado—, la función `get_request_settings` aplica una fusión profunda de un objeto `SeshatConfigOverride` opcional sobre el *singleton* base y devuelve un nuevo `SeshatConfig`. La fusión es recursiva a cualquier profundidad: un campo no presente en el *override* conserva su valor base en todos los niveles. La configuración resultante —global o por trabajo— gobierna el comportamiento de todas las etapas que se describen a continuación.

4.3. Ingesta

La ingesta comprende tres pasos: la validación del archivo de entrada, la transcripción de los archivos de audio y la normalización de ambas rutas de entrada en un `TranscriptDocument` común.

4.3.1. Validación

Los archivos de audio (.mp3, .wav, .m4a) pasan tres controles secuenciales antes de subirse a blob storage:

1. **Tamaño:** el archivo se rechaza si supera `max_file_bytes` (500 MiB por defecto).
2. **Duración:** se extrae con `mutagen` y se rechaza si supera `max_audio_seconds` (dos horas por defecto).
3. **Firma de bytes:** los primeros 12 bytes determinan el formato real (*magic bytes*), que se contrasta con la extensión declarada; cualquier discrepancia se rechaza. La extensión por sí sola no es fiable al controlarla el cliente.

El texto preformateado (.yaml, .yml, .json) solo requiere una comprobación de esquema y de coherencia de la fecha de reunión, tras la cual el campo `content` se extrae directamente como transcripción.

4.3.2. Transcripción

La transcripción de audio se delega en `AbstractTranscriber` —AssemblyAI por defecto—, envuelta en `TrackingTranscriber` para acumular los segundos consumidos contra el presupuesto de uso. El archivo original y la transcripción se persisten en blob storage como artefactos de auditoría independientes de la base de conocimiento; si el trabajo falla más adelante, ambos están disponibles para reintentar sin re-transcribir.

¹La referencia completa de parámetros y variables de entorno está disponible en <https://github.com/josepmafe/seshat/blob/main/docs/configuration.md>.

4.4. Extracción multi-agente en dos pasadas

La extracción es la fase de mayor coste computacional y el núcleo del sistema. Se articula en dos pasadas secuenciales con un contrato estricto: todos los agentes de la primera pasada deben completarse antes de que comience la segunda. Entre ambas se interpone la revisión humana opcional (HITL): la primera pasada identifica y puntúa los nodos, y sólo para los nodos aprobados —automática o manualmente— la segunda infiere las relaciones.

Todos los agentes comparten una base común que encapsula la invocación estructurada del LLM con reintentos, y se descubren en tiempo de ejecución a partir de la configuración; habilitar o deshabilitar un tipo de concepto es así un cambio de configuración y no de código. Sus prompts de sistema, estáticos por tipo, se marcan además como cacheables a nivel de proveedor para reducir coste y latencia. La arquitectura de clases de los agentes y el mecanismo de caché se detallan en el anexo B.

Las cuatro subsecciones siguientes describen respectivamente la primera pasada —identificación y puntuación de confianza—, y la segunda —recuperación y resolución—.

4.4.1. Identificación

La transcripción completa se envía en paralelo a cuatro agentes de identificación, uno por tipo de concepto (figura 4.2). El Minimum Viable Product (MVP) no realiza segmentación previa: la ventana de contexto de los modelos actuales es suficiente para transcripciones de reuniones típicas. Cada agente recibe además un *hint* ligero con los nodos de su mismo tipo ya presentes en la base de conocimiento —título y resumen de los más recientes, acotado en tokens—, lo que le permite ver ejemplos de nodos ya aprobados.

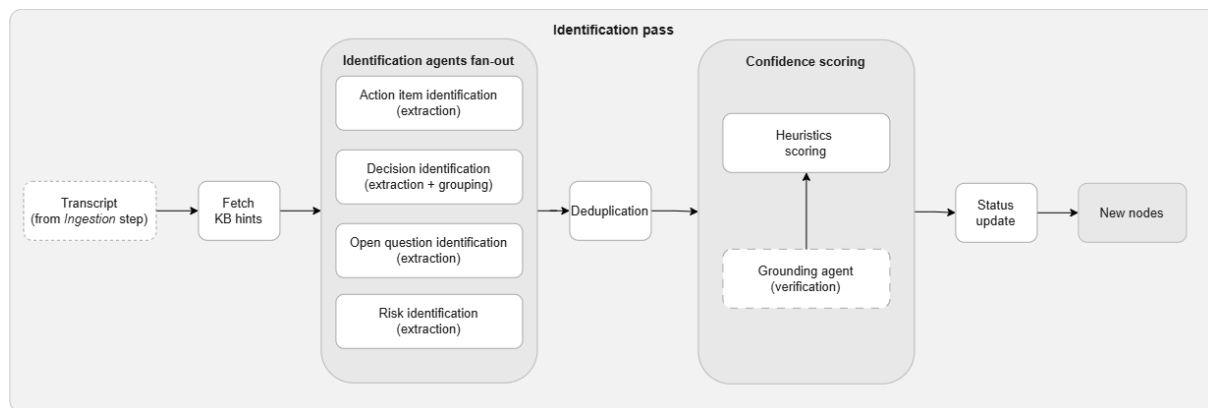


Figura 4.2.: Flujo de la primera pasada de extracción. La transcripción normalizada en la ingesta alimenta el *fan-out* de agentes de identificación —el de decisiones incluye además el paso de agrupación—; sus salidas se deduplican, se puntúan por confianza —heurísticas y, opcionalmente, verificación por *grounding*— y reciben un estado de aprobación (pasos descritos en la sección 4.4.2) antes de persistirse como nodos nuevos.

Cada agente devuelve una lista de nodos estructurados mediante salida forzada con esquemas Pydantic. Cada nodo queda anclado a una cita literal del texto fuente que lo respalda, lo que reduce la alucinación y hace auditable cada afirmación. El diseño de los prompts de identificación combina varias técnicas complementarias que se detallan en el anexo B.

Para el tipo **decision** se introduce un paso adicional de agrupación: la lista plana de conceptos se organiza en grupos temáticos, de modo que el *pipeline* genere un único nodo por decisión consolidada en lugar de múltiples nodos fragmentados. El paso es configurable por tipo, pues sólo las decisiones —que tienden a dispersarse a lo largo de un intercambio— se benefician de forma clara; ante un fallo del agrupador, cada concepto retrocede a un grupo unitario y ningún nodo se pierde.

Tras la ejecución de los agentes, los nodos con título idéntico y mismo tipo se fusionan (*deduplicación within-meeting*): el nodo más reciente en la salida del agente —que se asume representa el estado final de la deliberación— absorbe los anteriores.

Opcionalmente, cada agente de identificación puede envolverse en una segunda pasada de auto-revisión que valida y filtra sus propias salidas antes de puntuarlas; el mecanismo se describe en el anexo B.

4.4.2. Puntuación de confianza y estado de aprobación

Cada nodo recibe, antes de pasar a revisión, una puntuación de confianza en $[0, 1]$: una señal continua, resultado de la media ponderada de tres métricas heurísticas. Cada métrica aplica reglas léxico-sintácticas sobre el análisis morfológico, de dependencias y de entidades que produce el modelo `en_core_web_sm` de `spaCy`:

$$\text{confianza} = 0,3 \cdot s_{\text{cita}} + 0,3 \cdot s_{\text{título}} + 0,4 \cdot s_{\text{descripción}} \quad (4.1)$$

donde s_{cita} es una señal que satura a las 35 palabras de cita fuente. $s_{\text{título}}$ mide la especificidad del título combinando tres señales —presencia de entidades nombradas, calificadores y longitud, en ese orden de peso— que reflejan la importancia relativa de estas señales para distinguir enunciados específicos de generales [17]. $s_{\text{descripción}}$ penaliza multiplicativamente el lenguaje de cobertura, la voz pasiva y el tiempo futuro; los dos primeros son marcadores léxicos de incertidumbre ya empleados en la detección de *hedging* [18].

Opcionalmente, el **GroundingAgent** actúa como compuerta binaria independiente: un modelo ligero de un proveedor distinto al de los agentes de identificación —la configuración lo exige para evitar errores correlacionados— juzga si la descripción está respaldada por la cita fuente. Un nodo que falla la verificación es rechazado o marcado como pendiente de revisión, independientemente de su puntuación heurística.

La política de aprobación automática combina ambas señales: si la puntuación supera el umbral configurado y la verificación pasa (o está desactivada), el nodo se aprueba automáticamente con `approval_method=THRESHOLD`; en caso contrario el nodo queda en estado `AWAITING_REVIEW`. Con `auto_mode=True`, todos los nodos cuya puntuación supere el umbral establecido se aprueban sin revisión humana; los que no son rechazados automáticamente.

4.4.3. Recuperación aumentada

El módulo de recuperación (RAG) opera en la segunda pasada, una vez que los nodos de la primera han sido consolidados y aprobados. A diferencia del uso habitual de RAG, orientado a responder preguntas fundamentando la respuesta en los documentos previamente ingeridos, aquí la recuperación localiza los nodos preexistentes de la base de conocimiento con los que cada nodo nuevo podría relacionarse.

4. Implementación

Para cada nodo nuevo, el **NodeRetriever** construye una consulta a partir de su título y descripción, la envía al **SearchEngine** con el umbral de similitud configurado y expande los resultados con los vecinos directos del grafo (profundidad 1); por último, un reranking opcional reordena los candidatos por relevancia. Un presupuesto de contexto acotado en tokens limita el número total de candidatos para no sobrepasar la ventana del modelo de resolución. La figura 4.3 resume el recorrido de un nodo fuente a través del recuperador.

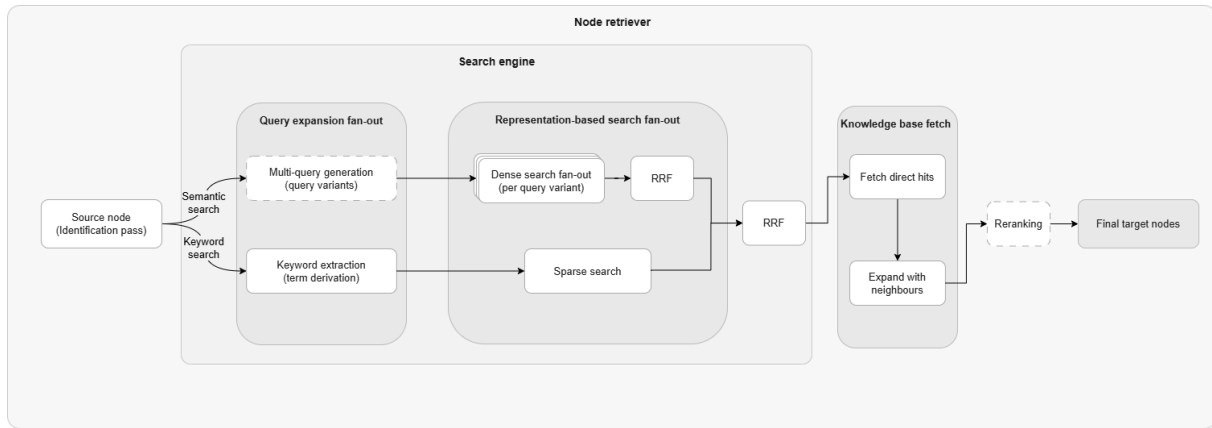


Figura 4.3.: Flujo de recuperación para un nodo fuente, en su configuración completa (modo HYBRID). El **SearchEngine** combina una pata semántica (búsqueda densa, con expansión *multi-query* opcional) y una pata dispersa por palabras clave, fundidas mediante RRF. El **NodeRetriever** obtiene los nodos de la base de conocimiento, los expande con sus vecinos directos y opcionalmente aplica un reranking (recuadro discontinuo) antes de devolver los nodos objetivo.

Un nodo puede recuperarse por dos vías complementarias, cada una con sus propias limitaciones. La búsqueda **densa** (*dense*) compara la consulta con los nodos en el espacio de *embeddings* y recupera por proximidad semántica: reconoce paráfrasis y sinónimos aunque no compartan vocabulario, pero tiende a difuminar los términos raros y específicos. La búsqueda **dispersa** (*sparse*) casa términos literales y sobresale justo donde la densa se diluye —nombres propios, herramientas o identificadores técnicos que el *embedding* promedia—, aunque no capta reformulaciones. **SearchEngine** expone ambas, y su combinación, en tres modos de búsqueda configurables:

- **SEMANTIC**: búsqueda densa por similitud coseno sobre pgvector. Es el modo por defecto. Opcionalmente admite un *fan-out* multi-consulta: un LLM genera variantes de la consulta original —más abstractas, más específicas o con distinto vocabulario— y sus resultados se funden con Reciprocal Rank Fusion (RRF) [19], ampliando la cobertura frente a una única formulación.
- **KEYWORD**: búsqueda dispersa. Un LLM ligero destila primero la consulta a 3–6 palabras clave discriminantes y la búsqueda se realiza sobre ellas; sin este paso, la coincidencia literal —que exige la presencia conjunta de todos los términos— se ahogaría en el vocabulario genérico de una consulta larga y devolvería poco o nada.
- **HYBRID**: combina la pata densa —con su *fan-out* multi-consulta si está activo— y la dispersa, fusionándolas mediante RRF, que las agrega por la posición de cada nodo en cada lista y no por su puntuación. Esto es deliberado: las similitudes coseno y las de texto viven en escalas distintas y no son directamente comparables, mientras que el rango sí lo es.

Sobre la lista devuelta por cualquiera de los modos puede aplicarse un *reranking* opcional. Mientras que la búsqueda densa aproxima la relevancia por la distancia entre vectores calculados de forma independiente para consulta y candidato, un modelo *cross-encoder* procesa ambos textos de forma conjunta y estima su relevancia directamente, lo que reordena los candidatos con mayor precisión a costa de una llamada adicional a un proveedor externo.

Por último, la recuperación es intencionalmente **multi-tipo**: un nodo de tipo **decision** puede necesitar resolver relaciones frente a nodos de tipo **risk** o **open_question**, de modo que restringir la búsqueda al mismo tipo privaría de candidatos válidos a los agentes de resolución cruzada.

4.4.4. Agentes de resolución

La resolución de relaciones opera en dos familias paralelas: **mismo tipo** y **tipo cruzado** (figura 4.4). En ambas, las relaciones son **dirigidas** y se infieren en un único sentido: desde cada nodo nuevo (fuente) hacia los nodos preexistentes de la base de conocimiento (destino) recuperados en la fase anterior. Los nodos de reuniones previas no se reevalúan entre sí; sólo la llegada de un nodo nuevo dispara la inferencia de relaciones, lo que mantiene el coste acotado al conocimiento incremental de cada trabajo.

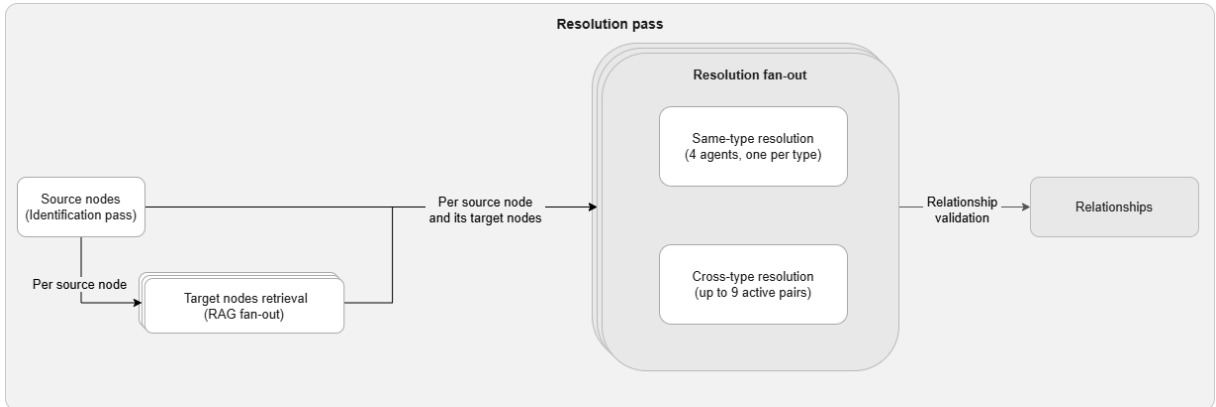


Figura 4.4.: Flujo de la segunda pasada. Para cada nodo fuente de la primera pasada se recuperan sus nodos objetivo (recuperación descrita en la sección 4.4.3); el par fuente-objetivos se envía en paralelo a las familias de resolución de mismo tipo (un agente por tipo) y de tipo cruzado (hasta nueve pares activos). Un paso de validación mediante reglas heurísticas filtra las relaciones inferidas antes de persistirlas.

Los agentes de mismo tipo clasifican cada nodo nuevo frente a los candidatos recuperados de la base de conocimiento en una de las relaciones aplicables a su tipo o sin relación. Por ejemplo, para un nodo de tipo **decision**, las relaciones permitidas del mismo tipo son **supersedes**, **amends** y **conflicts_with** (el conjunto completo por tipo se recoge en la tabla B.1 del anexo B). Además, el prompt establece criterios operativos precisos: **supersedes** requiere que la decisión previa quede funcionalmente obsoleta; **amends** implica que la decisión previa permanece vigente pero es matizada o restringida; **conflicts_with** requiere que ambas decisiones sean activas simultáneamente e incompatibles. Ante ambigüedad entre **amends** y **supersedes**, el sistema prefiere **amends** como clasificación menos destructiva.

4. Implementación

Los agentes de tipo cruzado detectan relaciones entre nodos de tipos distintos: `mitigates`, `blocks` y `resolves`. Por ejemplo, una `decision` puede mitigar (`mitigates`) un `risk` o resolver (`resolves`) una `open_question`. El esquema de relaciones restringe qué pares de tipos pueden evaluarse: de los 12 pares dirigidos posibles entre tipos distintos, el MVP habilita los 9 cuyas fronteras semánticas son más nítidas, materializados como instancias de agente fijas; los restantes quedan fuera de alcance por ser más ambiguos de clasificar de forma fiable. La tabla B.1 del anexo B detalla los pares habilitados.

Cuando un agente tiene incertidumbre genuina entre dos relaciones, puede señalarlo mediante el campo opcional `alt_rel_type`. Este campo es la interfaz entre el agente de resolución y una segunda pasada de auto-revisión opcional que arbitra los casos ambiguos, análoga a la de identificación y detallada igualmente en el anexo B.

Tras ambas llamadas paralelas, un paso de validación heurística descarta las relaciones malformadas —dirección incorrecta, incompatibilidades de tipo, violaciones del esquema— y las registra sin interrumpir el trabajo.

4.5. Orquestación y ciclo de vida del pipeline

Las dos pasadas de extracción descritas hasta aquí no se invocan directamente: son etapas de un *pipeline* que la API encola y un *worker* asíncrono ejecuta. Cada *pipeline* avanza por la máquina de estados descrita en la figura 3.3, desde su recepción hasta la persistencia de los resultados.

4.5.1. API y creación del pipeline

La API REST (FastAPI) es el único punto de entrada programático al sistema. Un *pipeline* se crea enviando un archivo de entrada, que la API valida y persiste junto con el registro del trabajo antes de encolarlo para su procesamiento asíncrono: la petición retorna de inmediato, sin esperar al resultado. El resto de operaciones del ciclo de vida —consultar estado, recuperar resultados, aprobar la revisión o reintentar un *pipeline* fallido— se exponen como *endpoints* sobre el mismo recurso; la referencia completa de la API se recoge en el anexo B.5. El `JobService` concentra esta lógica de dominio y se apoya en repositorios que encapsulan el acceso a datos, de modo que ningún *endpoint* consulta la base de datos directamente.

En la creación se aplican además dos salvaguardas: un control de duplicados por *hash* del contenido —que evita re-procesar un archivo ya ingerido salvo forzado explícito— y límites de tasa por usuario y de concurrencia global, que protegen el presupuesto de tokens frente a envíos masivos.

4.5.2. Worker asíncrono y máquina de estados

El *worker* consume los *pipelines* encolados y los conduce a través de los estados del ciclo de vida, actualizando el estado persistido en cada transición para que la API pueda reportarlo. La ejecución se divide en dos tramos separados por la revisión humana. El primero —ingesta, transcripción e identificación— termina dejando el *pipeline* en `AWAITING_REVIEW`. El segundo —recuperación, resolución y escritura— sólo arranca una vez aprobados los nodos. Cuando el modo automático está activo, o cuando ningún nodo queda pendiente de revisión, el segundo tramo se encadena sin intervención manual.

El *worker* está diseñado para tolerar fallos sin dejar la base de conocimiento en un estado inconsistente. La escritura final de nodos y relaciones es una única transacción, de modo que un fallo en cualquier punto no deja estado parcial. Además, cuando un *pipeline* falla queda en estado **FAILED** conservando su archivo original; la operación de reintento lo recupera del almacenamiento y vuelve a ejecutar el *pipeline* completo desde la transcripción, sin reutilizar el trabajo ya realizado —una reanudación consciente del progreso queda como trabajo futuro (sección 8.2)—. Por último, los *pipelines* interrumpidos por una caída del proceso durante la escritura se detectan y marcan como recuperables en el siguiente arranque.

4.5.3. Revisión humana (HITL)

En **AWAITING_REVIEW**, el *pipeline* se detiene a la espera de que un revisor decida el destino de los nodos extraídos. Las decisiones se envían en una única petición al *endpoint* de aprobación, que admite dos mecanismos combinables. El primero es una aprobación masiva por umbral: aprueba de una vez todos los nodos pendientes cuya confianza supere un valor dado. El segundo son decisiones individuales: aprobar, rechazar o editar un nodo concreto. Cuando se usan juntos, la regla masiva se aplica primero y las decisiones individuales se superponen a su resultado, de modo que el revisor puede, por ejemplo, aprobar en bloque todo lo que supere el umbral y luego rechazar o corregir los casos puntuales que lo requieran. Las ediciones y aprobaciones registran quién y cuándo, distinción que sostiene la trazabilidad descrita a continuación. Una vez enviadas las decisiones, el *pipeline* transita a la fase de resolución y el *worker* retoma el segundo tramo.

4.6. Mecanismos de seguridad

4.6.1. Control de acceso y autenticación

Todos los *endpoints* de la API requieren una clave de API en la cabecera **X-API-Key**. Las claves se almacenan como hashes bcrypt (factor de coste 12); el texto plano nunca se persiste. La validación de cada petición usa comparación en tiempo constante para prevenir ataques de temporización.

El modelo de roles es jerárquico; cada nivel hereda los permisos del anterior:

$$\text{viewer} \subset \text{reviewer} \subset \text{operator} \subset \text{admin}$$

viewer sólo consulta la base de conocimiento; **reviewer** es el nivel mínimo para enviar un *pipeline* y para decidir sobre los nodos en revisión —aprobarlos, rechazarlos o editarlos—. Activar el modo automático se sitúa también en este nivel, pues equivale a aprobar en bloque sin revisión manual. Las acciones más sensibles se reservan a niveles superiores: los *overrides* de configuración por trabajo requieren **operator**, mientras que las operaciones de borrado y la re-ingesta forzosa —que elimina nodos existentes— requieren **admin**. La provisión de claves de API queda al margen de esta jerarquía: se realiza a través de un router `/admin` separado, autenticado con una clave raíz (*root*) almacenada en Secrets Manager en lugar de con una clave de usuario.

4.6.2. Validación de entradas

Más allá del control de acceso, el sistema trata como hostiles sus dos entradas no controladas. El archivo del cliente se valida en la frontera de la API por firma de bytes (sección 4.3.1) en fallo cerrado: una entrada inválida se rechaza antes de crear el *pipeline* y no deja ni trabajo encolado ni artefacto almacenado. El texto de la transcripción, que llega a los agentes, se aísla frente a la inyección de prompts mediante los delimitadores XML del anexo B, reforzados por la salida estructurada forzada: aunque el modelo procese contenido malicioso, el resultado debe ajustarse al esquema Pydantic para no ser descartado.

4.7. Observabilidad y trazabilidad

La observabilidad es una preocupación transversal en un sistema construido sobre LLM: las llamadas a los modelos son costosas, no deterministas y ocurren en paralelo, por lo que sin instrumentación es difícil diagnosticar un fallo, atribuir un coste o justificar una afirmación extraída. **Seshat** aborda esto en tres planos: el trazado de la ejecución, la medición del consumo con enforcement de presupuesto, y la trazabilidad del conocimiento producido.

4.7.1. Trazado de la ejecución

MLflow 3 actúa como *backbone* de observabilidad. La instrumentación de las llamadas a los agentes es automática: `mlflow.langchain.autolog()` captura prompts, respuestas, uso de tokens y latencia sin código adicional en los agentes, emitiendo cada llamada como un *span* de OpenTelemetry (OTel), el estándar abierto de trazado distribuido. Cada *pipeline* abre un *run* de MLflow al arrancar; su identificador se expone en la consulta de estado del trabajo, de modo que el operador puede saltar directamente a la traza completa de la ejecución.

4.7.2. Medición del consumo y presupuesto

Sobre el trazado automático, **Seshat** añade una capa propia de medición que acumula, por etapa del *pipeline*, los tokens de entrada, salida y caché del LLM, los tokens de *embeddings* y de *reranking*, y los segundos de audio transcritos, junto con percentiles de latencia por llamada (media, p95 y máximo). Estas métricas se registran en MLflow y hacen comparables el coste y el rendimiento entre versiones del sistema.

Esta capa no es meramente informativa: aplica un presupuesto de consumo. Cada etapa acumula su uso contra unos límites configurables y aborta la ejecución si los rebasa, lo que acota el gasto ante transcripciones anómalas o bucles inesperados. El control de gasto real recae así sobre estos límites de tokens, y no sobre tablas de precios que requerirían mantenimiento; los costes monetarios se estiman de forma informativa en la interfaz de MLflow. Los detalles de implementación de esta capa —su propagación transparente a los agentes concurrentes y los envoltorios de medición— se describen en el anexo B.2.

4.7.3. Trazabilidad del conocimiento

Al margen de la instrumentación de ejecución, cada nodo conserva la traza de su origen y de su manipulación posterior. La cita literal del texto fuente (`quote_anchors`) que originó el nodo hace auditable cualquier afirmación extraída. Las correcciones humanas en revisión quedan registradas en los campos `corrected_by` y `corrected_at`, distinguiendo el contenido corregido del generado automáticamente, y las anulaciones (*overrides*) de operadores y administradores exigen justificación escrita, almacenada en `correction_reason`.

4.8. Despliegue e infraestructura

4.8.1. Contenedores Docker

El sistema se orquesta con Docker Compose en seis servicios: `postgres` (PostgreSQL 17 con pgvector), `mlflow` (observabilidad, con *backend* SQLite), `localstack` (emulación de S3 y Secrets Manager), `migrate` (aplica las migraciones de esquema mediante Alembic y termina), `seshat-api` (la API FastAPI, que aloja además el *worker* asíncrono en proceso) y `seshat-ui` (la interfaz de revisión en Streamlit). El arranque está ordenado por comprobaciones de salud: la API sólo se inicia cuando Postgres, LocalStack y MLflow están operativos y las migraciones han concluido.

La API y las migraciones se construyen desde un mismo *Dockerfile* multietapa —comparten la capa de dependencias y difieren sólo en la etapa final—, de modo que ambas ejecutan exactamente el mismo entorno. La configuración se inyecta por variables de entorno mediante el separador de anidamiento de pydantic-settings; los secretos —incluida la cadena de conexión de Postgres, bajo la clave `seshat/postgres_url`— se resuelven en el arranque desde Secrets Manager, de modo que el mismo código se ejecuta en desarrollo (LocalStack) y en producción (AWS real) sin ramas condicionales.

4.8.2. Integración continua

Cada *push* a la rama principal y cada *pull request* dispara una canalización de Continuous Integration (CI) (GitHub Actions) que actúa como guardián de calidad antes de la fusión. La canalización arranca con una etapa de *linting* —formato y reglas estáticas con *ruff*, más comprobación de tipos con *mypy*— de la que dependen las demás: si el estilo o los tipos fallan, no se malgasta cómputo en las pruebas. Superada esa puerta, las pruebas unitarias y las de integración (sección 6.1) se ejecutan en paralelo; estas últimas levantan los mismos contenedores de Docker Compose de la sección anterior, de modo que se validan contra Postgres, LocalStack y MLflow reales, y exigen una cobertura de ramas mínima del 80 %. Una última etapa, sólo en las *pull requests*, publica como comentario métricas de complejidad ciclomática y de mantenibilidad del código, dando visibilidad a la deuda técnica en el momento de la revisión.

La canalización ejecuta deliberadamente todas las pruebas salvo las marcadas como dependientes de un LLM en vivo: reproducir la aleatoriedad y el coste de las llamadas reales al modelo en cada *commit* no es ni fiable ni económico. La evaluación de la calidad de los agentes recae por ello en el *gate* descrito en el capítulo 5, que se ejecuta de forma deliberada fuera de la CI y cuyo veredicto la API comprueba en el arranque.

5. Evaluación del Sistema

5.1. Estrategia de evaluación

Evaluar un sistema construido sobre LLM plantea una dificultad propia: sus componentes no son deterministas —una misma entrada puede producir salidas distintas entre ejecuciones— y sus fallos son silenciosos, pues una alucinación tiene la misma forma que una extracción correcta. Sin una medición sistemática y repetible es imposible saber si un cambio de *prompt*, de modelo o de umbral mejora el sistema o lo degrada. La estrategia de evaluación de **Seshat** responde a esa dificultad con dos decisiones: medir por separado cada etapa no determinista, y convertir el resultado de esa medición en una condición de despliegue.

La calidad se mide con cinco arneses independientes, uno por cada etapa del *pipeline* con comportamiento no determinista: identificación, agrupación, *grounding*, recuperación (*retrieval*) y resolución de relaciones. La elección de esas cinco etapas no es arbitraria: el resto del *pipeline* es determinista y queda cubierto por las pruebas unitarias y de integración habituales, donde una salida incorrecta delata un error de código. Sólo estas cinco etapas, cuya salida depende de un LLM o de la similitud vectorial, pueden degradarse en silencio ante un cambio de *prompt*, de modelo o de corpus, sin que ninguna prueba convencional lo advierta; por eso son las que exigen medirse contra una referencia.

Cada arnés ejecuta el componente real del sistema contra un corpus de casos con verdad de referencia, y produce métricas de precisión y *recall* adaptadas a la tarea que evalúa, calculadas por *scorers* deterministas propios y no por un LLM como juez, para que el veredicto sea reproducible. Lo que se mide es así exactamente lo que se ejecuta en producción. Aislar cada etapa cumple además un segundo propósito, más allá de la cobertura: atribuir una regresión a su origen. Como la resolución opera sobre los candidatos que le entrega la recuperación, un fallo observado de principio a fin podría nacer en cualquiera de las dos; medirlas por separado, cada una contra su propia verdad de referencia, distingue si el problema está en *qué* se recuperó o en *cómo* se clasificó.

Los cinco arneses vuelcan su veredicto —una métrica por criterio, con su marca de aprobado o suspenso— a un fichero de *gate* común que agrega los resultados de todos los arneses ejecutados. Ese fichero no es un mero informe: la API se niega a arrancar si el *gate* no está presente o no ha pasado, de manera que el sistema no puede procesar datos reales con una calidad por debajo de los objetivos fijados. Los umbrales que definen ese aprobado residen en código, no en configuración, para que modificarlos exija un cambio revisable y no un simple ajuste de entorno.

La tabla 5.1 recoge el contrato completo de calidad: qué mide cada arnés, qué umbral debe superar cada métrica para aprobar y si su incumplimiento bloquea el arranque del sistema. No todas las métricas medidas condicionan el *gate*: algunas se registran en MLflow solo con fines de observabilidad, pues resultan demasiado estrictas o poco discriminantes para servir de criterio de bloqueo —una distinción que se justifica caso por caso en los resultados de cada arnés.

Arnés	Métrica	Umbral	Bloqueante
Identificación	precisión (por tipo)	≥ 0.75 – 0.85	Sí
	recall (por tipo)	≥ 0.75 – 0.85	Sí
	tasa espuria (por tipo)	≤ 0.10 – 0.15	Sí
	F1	—	No
Agrupación	<code>group_hit_rate</code>	≥ 0.80	Sí
	<code>exact_match</code>	—	No
<i>Grounding</i>	precisión	≥ 0.85	Sí
	recall	≥ 0.80	Sí
Recuperación	<code>recall@5</code>	≥ 0.70	Sí
	<code>mrr@5</code>	≥ 0.75	Sí
	<code>precision@5</code>	—	No
Resolución	precisión (por tipo)	≥ 0.80	Sí
	recall (por tipo)	≥ 0.80	Sí
	F1	—	No

Tabla 5.1.: Criterios del *gate* de evaluación: métricas medidas por cada arnés, umbral de aprobado y si su incumplimiento impide el arranque del sistema. Los umbrales de identificación varían según el tipo de concepto; se indica su rango y los valores exactos por tipo se recogen en la tabla 5.3.

La arquitectura interna de los arneses —el emparejador *greedy* de identificación, el mecanismo de caché compartido y el diseño de las trazas de MLflow— se detalla en el anexo B.3.1; este capítulo se centra en qué se mide, con qué corpus y con qué resultado.

5.2. Corpus sintético de evaluación

En ausencia de transcripciones reales de reuniones —por razones de privacidad y disponibilidad— el corpus de referencia se genera con apoyo de un LLM: cada fichero YAML describe un escenario de reunión sintético junto con la salida esperada del componente bajo prueba. Un corpus curado a partir de datos reales sería preferible, pero no es viable en esta fase del proyecto. A pesar de ello, todos los casos se diseñan y revisan manualmente para compensar parcialmente esta limitación.

Cada arnés organiza su corpus en *tiers*: familias de casos que comparten un mismo propósito de prueba y cuyo nombre de fichero lleva el *tier* como prefijo explícito, seguido de un número de secuencia y de una descripción breve del escenario. Así, un fichero como `boundary_001_decision_coexists_with_risk.yaml` revela sin abrirlo tanto su rol como la ambigüedad que ejercita: una decisión y un riesgo que conviven en el mismo fragmento. Esta convención permite auditar de un vistazo la cobertura del corpus y añadir casos nuevos sin romper su organización. La tabla 5.2 resume la composición actual:

Los *tiers negative* y *null* merecen una mención aparte: comprueban que el sistema no extrae ni resuelve nada cuando no corresponde, un aspecto que la precisión sólo capta indirectamente. Los *boundary* y *adversarial*, por su parte, documentan dentro del propio fichero por qué un caso es ambiguo; esa nota sobrevive a los cambios de *prompt* y evita que una simple oscilación de puntuación se confunda con una regresión real.

Tier	Rol del caso	Casos
Identificación		31
happy	Transcripciones claras y cooperativas; la extracción debe acertar con alta confianza	10
boundary	Interacciones entre tipos donde el agente tiende a alucinar o a suprimir de más	8
negative	Uno o más agentes deben devolver vacío; ejercita los criterios de rechazo	7
adversarial	Trampas de inyección de <i>prompt</i> y de atribución de responsable	3
realistic	Transcripciones largas y naturales, con deriva temática y varios locutores	3
Resolución		44
simple	Un nodo fuente y uno de la base: una única relación esperada evidente, o ninguna	23
null	Salida esperada sin relaciones; nodos temáticamente próximos pero no vinculados	11
multi	Un nodo fuente frente a varios candidatos; algunos relacionados, otros no	7
realistic	Contexto completo de reunión: varios nodos fuente y de la base, mezcla de tipos	3
Recuperación		40
same_type	<i>Pool</i> de un solo tipo; consulta y candidato del mismo tipo	4
cross_type	<i>Pool</i> de un solo tipo; consulta y candidato de tipos distintos	9
realistic	<i>Pool</i> mixto de los cuatro tipos; distribución realista de producción	17
negative	<i>Pool</i> mixto sin candidatos relevantes; nada debería situarse arriba	10
Agrupación		8
singleton	Cada ítem debe formar su propio grupo; no hay fusión	2
merge	Varios ítems que deben fundirse en un único grupo	2
mixed	Combinación de ítems que se fusionan y de ítems que quedan sueltos	2
realistic	Transcripciones largas; agrupación de extremo a extremo	2
Grounding		12
faithful	La cita respalda genuinamente la afirmación; el agente no debe rechazar de más	5
hallucination	La afirmación añade hechos ausentes en la cita; el agente debe detectar la fabricación	6
adversarial	Trampas de inyección de <i>prompt</i> y de manipulación de identidad	1

Tabla 5.2.: Composición del corpus de evaluación por arnés y *tier*, con el rol que cumple cada *tier* y su número de casos.

5.3. Métricas y resultados por arnés

Los resultados de esta sección corresponden a la última ejecución del *gate* sobre el corpus descrito (`eval_gate.json`). Las tablas recogen los valores medidos; el umbral que cada métrica debe superar y su carácter bloqueante figuran en el contrato de la tabla 5.1. En identificación, cuyos umbrales varían por tipo de concepto, se repiten junto a cada valor para facilitar la comparación directa.

Todos los arneses puntúan con *scorers* deterministas propios, sin recurrir a un LLM como juez: solapamiento difuso de citas y campos en identificación, comparación de conjuntos en agrupación, matrices de confusión en *grounding*, métricas de posición en *retrieval* y coincidencia exacta de ternas en resolución. La decisión es deliberada. Un juez basado en LLM reintroduciría en la propia medición el no determinismo y potenciales alucinaciones, y volvería el veredicto irreproducible; un *scorer* determinista, en cambio, produce la misma puntuación ante la misma predicción.

El no determinismo residual proviene entonces del *pipeline* evaluado, no del instrumento de medida: incluso a temperatura cero, la salida del LLM puede variar entre ejecuciones por la no asociatividad de la aritmética en coma flotante bajo ejecución en paralelo [20]. Que el instrumento sea determinista es condición necesaria para que ese ruido no se confunda con el del sistema, para que el *gate* sea un criterio estable y para el requisito de evaluación funcionalmente determinista del capítulo 3.

5.3.1. Identificación

El arnés de identificación ejecuta la primera pasada de extracción sobre la transcripción de cada caso y empareja los nodos extraídos con los esperados para contarlos como aciertos, falsos positivos o falsos negativos; el emparejador que decide esas correspondencias se describe en el anexo B.3.1. De ese recuento salen la precisión y el *recall* por tipo de concepto. Junto a ellos se registra la **tasa espuria**, que mide un fallo distinto: la extracción de un tipo de concepto que no debería aparecer en el caso. Se calcula sobre los casos donde un tipo está ausente de la verdad de referencia, como la fracción de ellos en los que el agente extrajo al menos un nodo de ese tipo; captura así la contaminación entre tipos que la precisión, calculada por tipo, no refleja.

Tipo	Precisión	Recall	Tasa espuria
<code>decision</code>	0.964 (≥ 0.80)	0.893 (≥ 0.80)	0.000 (≤ 0.10)
<code>risk</code>	0.850 (≥ 0.75)	1.000 (≥ 0.80)	0.000 (≤ 0.10)
<code>action_item</code>	0.895 (≥ 0.85)	0.947 (≥ 0.85)	0.077 (≤ 0.10)
<code>open_question</code>	0.867 (≥ 0.75)	0.867 (≥ 0.75)	0.111 (≤ 0.15)

Tabla 5.3.: Precisión, recall y tasa espuria de identificación por tipo de concepto. Cada valor está promediado sobre los casos del corpus.

Los cuatro tipos superan su umbral. El umbral de tasa espuria de `open_question` es deliberadamente más laxo (0.15 frente a 0.10): la frontera entre una pregunta abierta y un riesgo o una decisión pendiente es genuinamente ambigua en el lenguaje conversacional, y así queda reflejado también en su tasa espuria observada, la más alta de los cuatro tipos.

5.3.2. Agrupación

La agrupación es el paso que consolida las decisiones fragmentadas de la primera pasada en un único nodo por decisión (sección 4.4.1). El agente recibe la lista plana de conceptos extraídos y la reparte en grupos, cada uno destinado a fundirse en un solo nodo, de manera que su salida es una partición P de esos conceptos que se contrasta con la partición de referencia E . Ambas se tratan como conjuntos de grupos, y cada grupo como un conjunto de conceptos, por lo que ni el orden de los grupos ni el de los conceptos dentro de cada grupo influye en la puntuación. Sobre ellas se definen dos métricas, que comparan las particiones a distinta resolución:

$$\text{exact_match} = \begin{cases} 1 & \text{si } P = E, \\ 0 & \text{en caso contrario,} \end{cases} \quad \text{group_hit_rate} = \frac{|P \cap E|}{|E|}. \quad (5.1)$$

La primera es una medida de todo o nada: vale 1 sólo si la partición predicha coincide íntegramente con la esperada y cae a 0 en cuanto un solo grupo difiere. La segunda concede crédito parcial: es la fracción de grupos esperados que reaparecen exactamente en la predicción.

Métrica	Valor
<code>exact_match</code>	0.750
<code>group_hit_rate</code>	0.850

Tabla 5.4.: Resultados del arnés de agrupación. Cada valor está promediado sobre los casos del corpus.

Sólo `group_hit_rate` condiciona el *gate*; `exact_match` se registra pero no bloquea, por ser demasiado estricta a medida que crece el número de grupos por caso: un único grupo mal formado hunde toda la puntuación del ejemplo aunque el resto sea correcto. La brecha entre ambos valores observados —0.750 frente a 0.850— es precisamente esa penalización: el agente acierta la mayoría de los grupos, pero rara vez reproduce la partición entera sin un solo desajuste, y es `group_hit_rate` quien recoge ese acierto parcial que `exact_match` descarta.

5.3.3. Grounding

El *grounding* es la compuerta opcional que, durante la puntuación de confianza, verifica si la cita de un nodo respalda su título y descripción (sección 4.4.2). Su arnés mide si el agente emite ese juicio correctamente: cada nodo del corpus lleva anotado si su cita lo respalda o no, y la predicción del agente se clasifica en la matriz de confusión (verdaderos y falsos positivos y negativos) que se agrega en precisión y *recall*.

Métrica	Valor
Precisión	1.000
Recall	1.000

Tabla 5.5.: Resultados del arnés de *grounding*. Ambos valores se agregan sobre todos los nodos del corpus en una única matriz de confusión.

El resultado perfecto debe leerse con cautela: el corpus de *grounding* es de los más pequeños de los cinco (12 casos), de modo que el 1.000 significa «sin fallos observados en 12 casos», no una garantía de perfección; un único caso adicional podría desplazar la métrica de forma apreciable.

5.3.4. Recuperación (retrieval)

Los tres arneses anteriores cubren la primera pasada de extracción; los dos siguientes evalúan la segunda, que arranca recuperando los nodos con los que cada nodo nuevo podría relacionarse (sección 4.4.3). El arnés de recuperación siembra el almacén vectorial con los candidatos de cada caso y comprueba si la búsqueda sitúa los nodos relevantes entre los cinco primeros resultados.

Es también la etapa más configurable del *pipeline*: el modo de búsqueda, los modelos de lenguaje de sus patas densa y dispersa —con sus propios parámetros— y el modelo de *embeddings* son todos intercambiables (sección 4.4.3), y cada combinación desplaza el equilibrio entre latencia, coste y calidad. Las cifras de esta sección no son, por tanto, un techo del componente, sino el resultado de una configuración concreta, elegida por ofrecer un buen compromiso entre velocidad y rendimiento.

Métrica	Valor
recall@5	0.742
precision@5	0.560
mrr@5	0.944

Tabla 5.6.: Resultados del arnés de recuperación. Cada valor está promediado sobre los casos del corpus.

El corpus mezcla deliberadamente candidatos de los cuatro tipos de concepto en un mismo *pool* para simular la distribución realista de un almacén vectorial en producción; un *pool* de un solo tipo suprimiría falsos positivos y produciría una precisión artificialmente alta. Esta elección explica en parte por qué **precision@5** es sensiblemente más baja que **recall@5** y **mrr@5**: el sistema puede devolver candidatos plausibles de otro tipo entre los cinco resultados sin que ello indique un fallo de recuperación del nodo relevante, que **mrr@5** confirma que aparece consistentemente en una posición temprana.

5.3.5. Resolución de relaciones

La resolución es el paso final de la segunda pasada: clasifica la relación entre cada nodo nuevo y los candidatos recuperados. Su arnés ejecuta esa clasificación y compara las relaciones inferidas con las esperadas exigiendo coincidencia exacta de la terna (nodo origen, nodo destino, tipo de relación): una relación sólo cuenta como acierto si coinciden los tres elementos. El criterio es estricto y no gradúa la gravedad del error, pues todos los desaciertos pesan lo mismo. Por ejemplo, entre las relaciones que un mismo agente puede asignar a un par de decisiones, confundir **amends** con **supersedes** es un error leve, ya que ambas describen matices de una misma sustitución. Pero tomar por **amends** lo que era **conflicts_with** es mucho más grave, ya que invierte la compatibilidad entre ambas decisiones: dar por matización armónica lo que en realidad es una contradicción activa entre ellas.

Para medir la resolución de forma aislada, el corpus fija a mano el conjunto de candidatos de cada caso en lugar de tomarlo de la recuperación real: el arnés evalúa así la decisión de resolución suponiendo una recuperación perfecta, no la calidad de principio a fin.

Tipo de origen	Precisión	Recall
<code>decision</code>	0.910	0.962
<code>risk</code>	0.944	1.000
<code>action_item</code>	0.889	1.000
<code>open_question</code>	1.000	1.000

Tabla 5.7.: Precisión y recall de resolución por tipo de concepto origen. Cada valor está promediado sobre los casos del corpus.

Los cuatro tipos de origen superan con holgura el umbral, pero conviene leer estas cifras a la luz del supuesto anterior: miden la calidad de la decisión de resolución en condiciones ideales de recuperación, un límite superior que el rendimiento de principio a fin no necesariamente alcanza.

La tabla anterior desglosa el resultado por tipo de concepto origen, pero no es el único corte posible: los registros agentes de mismo tipo y tipo cruzado (sección 4.4.4) son independientes, cada uno con su propio conjunto de *prompts*: mejorar uno no afecta al otro, así que separar sus resultados permite atribuir un fallo a la familia responsable en lugar de diluirlo en un promedio conjunto. Un mismo caso del corpus puede combinar relaciones resueltas por ambos registros, de modo que la tabla 5.8 puntúa a nivel de relación individual, reprocesando los resultados cacheados del arnés.

Tipo de origen	Familia	Precisión	Recall
<code>decision</code>	Mismo tipo	0.857	0.967
	Tipo cruzado	0.875	0.875
<code>risk</code>	Mismo tipo	0.917	1.000
	Tipo cruzado	1.000	1.000
<code>action_item</code>	Mismo tipo	0.879	1.000
	Tipo cruzado	0.667	1.000
<code>open_question</code>	Mismo tipo	1.000	1.000
	Tipo cruzado	1.000	1.000

Tabla 5.8.: Precisión y recall de resolución por tipo de concepto origen y familia de agente, puntuados a nivel de relación individual según los tipos de concepto de su nodo origen y destino.

El desglose confirma que la resolución de tipo cruzado no es intrínsecamente más difícil que la de mismo tipo pese a tener más pares origen-destino posibles: en tres de los cuatro tipos de origen —`risk`, `action_item` y `open_question`— el recall de tipo cruzado iguala o supera al de mismo tipo. La única caída notable de recall es `decision`, que pierde 9 puntos al pasar de mismo a tipo cruzado. El caso opuesto —`action_item` en tipo cruzado, con la precisión más baja de toda la tabla (0.667) pero recall perfecto— corresponde a muy pocas relaciones (3 casos): el agente no omite ninguna relación esperada, pero inventa alguna adicional, un patrón distinto al de `decision` y consistente con la limitación de la coincidencia exacta de terna señalada en la sección 5.5.

5.4. Calibración de umbrales

Los arneses anteriores miden el sistema con sus umbrales ya fijados, pero esos umbrales no se eligen a mano. Dos señales continuas del *pipeline* gobiernan decisiones binarias a través de un corte, y fijar ese corte por intuición sería arbitrario y frágil: la confianza que determina si un nodo identificado se aprueba automáticamente o se envía a revisión humana, y la puntuación de similitud que filtra los resultados de *retrieval*. Ambas se calibran mediante un barrido de umbral contra el corpus de referencia, buscando el valor que optimiza una métrica objetivo. La calibración cierra así el ciclo con las secciones anteriores: los mismos arneses que miden la calidad proveen también los datos con los que se ajustan los parámetros que la gobiernan.

Ese barrido es barato porque reutiliza la caché de predicciones de los arneses correspondientes: recalcular el umbral óptimo recorre en memoria resultados ya calculados, sin repetir ninguna llamada al LLM (anexo B.3.2). Las dos subsecciones siguientes detallan qué métrica objetivo guía cada barrido, pues difieren según lo que cada umbral pone en juego.

5.4.1. Umbral de confianza (identificación)

El umbral sobre la puntuación de confianza heurística decide qué nodos identificados se aprueban sin intervención humana y cuáles se envían a la revisión (*HITL*). Un nodo por debajo del umbral no se descarta, sino que se encola para revisión: sólo cambia de canal de aprobación. Por eso F1 no sirve aquí, ya que contaría ese nodo diferido como un falso negativo.

La calibración se guía en cambio por dos magnitudes complementarias, medidas sobre el conjunto de nodos auto-aprobados: la **precisión** —qué fracción de ellos eran correctos— y la **cobertura** —qué fracción del total de nodos esperados llegan a auto-aprobarse—. Ambas tiran en sentidos opuestos: subir el umbral aprueba menos nodos pero más fiables, mejorando la precisión a costa de la cobertura. El barrido resuelve esa tensión eligiendo el umbral t^* de mayor cobertura entre los que aún satisfacen una precisión objetivo p_{obj} (0.95 por defecto):

$$t^* = \arg \max_t \text{cobertura}(t) \quad \text{sujeto a} \quad \text{precisión}(t) \geq p_{\text{obj}}. \quad (5.2)$$

Si ningún umbral alcanza p_{obj} , se conserva el de mayor precisión. El umbral resultante gradúa así cuánta carga de revisión se traslada al operador sin comprometer la fiabilidad de lo que se aprueba solo.

La figura 5.1 muestra el barrido sobre el corpus de identificación y hace visible esa tensión: al elevar el umbral, la cobertura desciende de forma monótona mientras la precisión asciende. Con $p_{\text{obj}} = 0,95$, el umbral seleccionado es $t^* = 0,76$ —el de mayor cobertura (0,60) entre los que superan la precisión objetivo—, marcado en la figura.

La señal que se calibra es puramente heurística. Se contempló complementarla con una segunda señal basada en NLI, que verificara si la descripción de un nodo se deduce de su cita fuente, pero su incorporación se ha aplazado (capítulo 8): exigiría un modelo adicional y añadiría latencia por nodo, y queda por confirmar que aporta información realmente ortogonal a las heurísticas. La calibración opera, por tanto, sobre el único corte heurístico vigente.

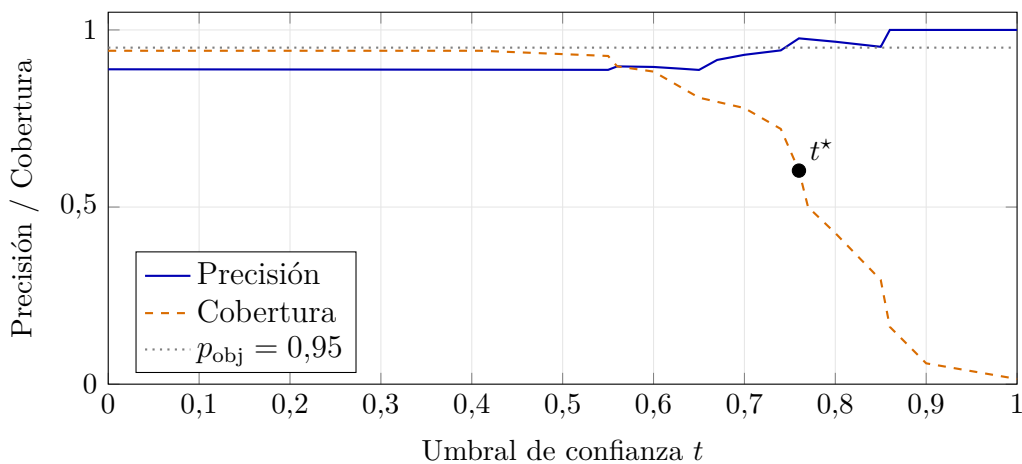


Figura 5.1.: Barrido del umbral de confianza sobre el corpus de identificación. La cobertura (fracción de nodos auto-aprobados) decrece al subir el umbral, mientras la precisión sobre los auto-aprobados crece. El punto $t^* = 0,76$ es el de mayor cobertura entre los que alcanzan la precisión objetivo $p_{\text{obj}} = 0,95$.

5.4.2. Umbral de similitud (retrieval)

El umbral de similitud descarta los candidatos de *retrieval* demasiado poco relevantes antes de pasarlos a la resolución, y se calibra de forma independiente por modo de búsqueda, ya que la escala de puntuación no es comparable entre modos: similitud coseno en el modo semántico, relevancia textual en el de palabras clave y RRF en el híbrido. El barrido maximiza el **F2 macro-promediado** sobre el corpus: cada caso positivo aporta su F2 —que pondera el *recall* el doble que la precisión, pues omitir un nodo relevante priva a la resolución de un candidato válido, un error más costoso que arrastrar uno irrelevante— y cada caso negativo aporta su especificidad —1 si no devuelve nada por encima del umbral, 0 si cuela algún irrelevante—, de modo que un umbral degenerado en cero, que lo aceptaría todo, no pueda dominar la selección.

5.5. Análisis de errores y limitaciones

Los resultados anteriores deben interpretarse junto con las limitaciones conocidas de cada arnés:

- **Identificación:** el emparejamiento por solapamiento difuso de citas puede aceptar como válida una cita que en realidad pertenece a una frase distinta pero suficientemente parecida; además, sólo se puntúan los campos estructurados de cada nodo, no la calidad semántica de su título y su descripción.
- **Resolución:** dos limitaciones ya señaladas se combinan —la coincidencia exacta de terna no distingue un error grave de uno leve, y el supuesto de recuperación perfecta—, por lo que sus cifras son un límite superior optimista.
- **Recuperación:** sólo evalúa la recuperación basada en *embeddings*, pero no el efecto del *reranking* posterior y de la expansión por vecinos del grafo.
- **Agrupación y *grounding*:** con 8 y 12 casos son los corpus más pequeños de los cinco, lo que limita la confianza estadística de sus métricas frente a los de identificación, resolución y recuperación.

6. Pruebas y Validación de Producto

Este capítulo cubre cómo se testea **Seshat** y cómo se pone en manos de sus usuarios. La comprobación se apoya en dos niveles complementarios: una suite de pruebas automatizadas —unitarias y de integración— verifica que el código se comporta como se espera, y el *gate* de evaluación (capítulo 5) protege la calidad de los agentes frente a regresiones. El tercer elemento es de otra naturaleza: una interfaz web que ofrece a los usuarios una forma sencilla de operar el sistema —enviar reuniones, revisar los nodos extraídos y explorar el grafo— sin tener que construir a mano las peticiones, extensas y anidadas, que la API espera y necesita. Este capítulo recorre los tres.

6.1. Tests unitarios y de integración

La suite de pruebas separa lo que puede verificarse de forma aislada y barata de lo que exige levantar sistemas externos. Esa separación no es sólo organizativa: gobierna qué se ejecuta en cada contexto —en cada *commit*, en cada *pull request* o sólo bajo demanda— y es la base de la canalización de integración continua descrita en la sección 4.8.2.

6.1.1. Pruebas unitarias

Las pruebas unitarias cubren la lógica pura y los adaptadores con todo el mundo exterior sustituido por *mocks*: el cálculo de confianza heurística, los validadores de entrada, la fusión de configuración, los *scorers* de evaluación, la lógica de los *routers* de la API —que se prueban sin levantar un servidor, sustituyendo las dependencias inyectadas y ejercitando la aplicación en proceso— o el comportamiento base de los agentes. Son rápidas y deterministas, no necesitan credenciales ni servicios, y constituyen el grueso de la suite. Los *mocks* se apoyan en las interfaces abstractas que aíslan a **Seshat** de sus proveedores: los LLM se sustituyen por *mocks* que devuelven salidas estructuradas predefinidas, y componentes como el *reranker* cuentan con implementaciones falsas de su interfaz para probar la lógica que los rodea.

6.1.2. Pruebas de integración

Las pruebas de integración ejercitan el código contra las dependencias reales —PostgreSQL con pgvector, el almacenamiento S3 y Secrets Manager emulados por LocalStack, y MLflow— levantadas mediante los mismos contenedores de Docker Compose que se usan en desarrollo, sin réplicas ni simuladores de esas dependencias. Cada prueba parte de un estado limpio: la base de datos de prueba se crea y migra una vez, y el estado se reinicia entre pruebas vaciando las tablas y recreando la colección vectorial. No todas requieren una clave de un proveedor de pago: la persistencia en pgvector, por ejemplo, se valida con un modelo de *embeddings* determinista que evita llamar a un servicio real.

El eje que estructura estas pruebas es un sistema de *marcadores* de `pytest` organizados de forma jerárquica: `integration` engloba toda prueba que necesita un servicio externo, y dentro de ella `llm` —y sus subconjuntos `agents`, `embedding`, `transcription` y `reranker`— señala las que requieren una clave de un proveedor en vivo. Por defecto, la configuración de `pytest` excluye el marcador `llm`, de modo que la ejecución habitual es rápida y gratuita; las pruebas que consumen llamadas de pago a los modelos son explícitamente opcionales. Este diseño responde a una tensión real: las pruebas más valiosas para un sistema basado en LLM son también las más lentas, caras y no deterministas, y forzar su ejecución en cada *commit* sería inviable.

Las pruebas dependientes de un servicio están protegidas además por una doble compuerta: al marcador se suma una condición de salto (`skipif`) que sondea la disponibilidad real del servicio o de la clave. Así, un mismo fichero de prueba se comporta correctamente tanto en la integración continua —con los servicios levantados— como en una máquina de desarrollo sin credenciales, donde simplemente se omite en lugar de fallar. Para las pruebas que sí necesitan un modelo, un ayudante selecciona automáticamente el proveedor disponible más económico, lo que hace la suite portable a cualquier conjunto de credenciales del ejecutor.

La prueba de mayor alcance es un *golden path* de extremo a extremo que recorre el *pipeline* completo —ingesta, transcripción, identificación y resolución— sobre una transcripción breve, con Postgres, LocalStack y un modelo real: la aproximación más cercana a una prueba de sistema.

6.2. Eval gate y regresión automatizada

Las pruebas de la sección anterior verifican el comportamiento del código, pero no bastan para un sistema cuyo núcleo es un modelo de lenguaje: un cambio de *prompt*, de modelo o de corpus puede degradar la calidad de la extracción sin romper ninguna aserción. Esa clase de regresión es justamente la que el *gate* de evaluación vigila. Sus cinco arneses, su corpus y sus umbrales se describen en el capítulo 5; lo que aquí importa es su papel como mecanismo de control de regresión.

El *gate* funciona como una prueba de aceptación de la calidad de los agentes, no del código. Al ejecutarse contra el corpus de referencia produce un veredicto agregado —aprobado o suspenso frente a los umbrales fijados— que se persiste en un fichero versionable. Comparar ese fichero entre dos ejecuciones convierte la calidad en una magnitud observable: si una modificación hace caer la precisión de un tipo de concepto por debajo de su umbral, el *gate* pasa a suspenso y la regresión se detecta antes de llegar a producción. Ese veredicto tiene consecuencias operativas: la API comprueba el *gate* en el arranque y se niega a servir si no está en verde (sección 4.5), de modo que la calidad medida es una precondition del despliegue, no un informe que se pueda ignorar.

A diferencia de las pruebas unitarias y de integración, el *gate* no forma parte de la canalización de integración continua: reproducir sus llamadas reales a los modelos en cada *commit* sería caro y no determinista, por las mismas razones que apartan a las pruebas `llm` de la ejecución por defecto. Se ejecuta, por tanto, de forma deliberada —antes de una fusión sensible, de un cambio de modelo o de un despliegue— y su resultado se conserva junto al código como constancia de la calidad con la que se validó cada versión.

6.3. Experiencia de usuario (UX)

La API REST es el único punto de entrada al sistema, pero operarla directamente exige construir peticiones HTTP a mano —subir un archivo por *multipart*, componer el cuerpo JSON de una revisión con las decisiones de cada nodo, encadenar consultas para recorrer el grafo—. Para que un usuario pueda trabajar con **Seshat** sin ese esfuerzo, el proyecto incluye una interfaz web: una herramienta de conveniencia, una capa de ergonomía sobre la API que traduce esas operaciones en formularios, botones y vistas, y hace el sistema utilizable en la práctica sin conocer la API.

Su rasgo de diseño más relevante es precisamente lo que *no* hace. La interfaz se comunica con el sistema exclusivamente a través de la API: no accede a la base de datos, al almacenamiento de objetos ni a ningún servicio interno, y no reimplementa ninguna regla del *pipeline*. La aprobación automática, la puntuación de confianza, la resolución de relaciones y las transiciones de estado ocurren todas en el servidor; la interfaz sólo dispara acciones y representa lo que la API le devuelve. Es, además, sin estado: se autentica introduciendo una clave de API —la misma que gobierna los roles (sección 4.5)— y no persiste nada entre sesiones. Construida sobre Streamlit, esta delegación estricta la mantiene como un cliente fino que no puede divergir del comportamiento del sistema.

Sobre esa base, la interfaz cubre el ciclo de vida completo de un *pipeline*. Un operador envía un archivo desde un formulario y sigue su avance por la máquina de estados en una barra de progreso que sondea periódicamente el estado del trabajo; no gobierna las transiciones —de eso se encarga el *worker*— sino que las refleja. Cuando el trabajo se detiene en revisión, presenta la interfaz de HITL descrita en el capítulo 4: aprobación masiva por umbral de confianza y decisiones individuales de aprobar, rechazar o editar cada nodo, con acceso al fragmento de transcripción que respalda cada uno. Fuera del flujo de revisión, ofrece acciones manuales —añadir nodos o relaciones, corregir contenido mediante anulaciones justificadas— y herramientas de exploración del grafo: consulta por filtros, búsqueda semántica y una vista interactiva del impacto de un nodo sobre sus vecinos. Un panel de administración, protegido por la clave raíz, gestiona las claves de API. En conjunto, la interfaz hace visible y operable desde el navegador lo que la API expone de forma programática, sin alterar lo que ya se ha descrito en los capítulos anteriores.

7. Discusión

7.1. Lecciones aprendidas

El desarrollo de **Seshat** dejó varias lecciones que trascienden este proyecto concreto y que orientarían el diseño de cualquier sistema similar basado en LLM.

- **La evaluación reproducible es innegociable, pero no es suficiente por sí sola.** En un sistema no determinista, sin una medición repetible es imposible saber si un cambio mejora o degrada la calidad; por eso el *gate* descansa sobre *scorers* deterministas y no sobre un juez basado en LLM. Ahora bien, esos *scorers* sólo alcanzan los aspectos estructurales y dejan sin medir dimensiones cualitativas —la calidad semántica de un título o una descripción, por ejemplo (sección 5.5)—. Ahí un juez basado en LLM complementaría bien la evaluación determinista, siempre como señal observacional y no como criterio de bloqueo, para no reintroducir en el veredicto el no determinismo que se quiere contener.
- **Frente a un objetivo móvil, el *prompt engineering* sobre una abstracción de proveedor supera al *fine-tuning*.** Los modelos base cambian cada pocos meses; apostar por agentes guiados por *prompt* tras una interfaz de proveedor común mantuvo el sistema adaptable —cambiar de modelo o de proveedor es configuración, no reentrenamiento— y evitó los costes de datos y cómputo que un modelo ajustado habría atado a una versión concreta.
- **La fiabilidad nace de superponer salvaguardas modestas, no de una sola fuerte.** Ninguna medida contra la alucinación es infalible, pero el anclaje en cita, la auto-revisión opcional y la revisión humana guiada por confianza se cubren mutuamente sus puntos ciegos. Este principio de defensa en profundidad se desarrolla en la sección 7.2.
- **Aislar cada etapa se pagó en evaluabilidad.** Las fronteras entre componentes no se trazaron sólo por limpieza arquitectónica, sino para poder medir cada etapa por separado. Ese aislamiento es lo que permite un arnés por etapa y atribuir una regresión a su origen —qué se recuperó frente a cómo se clasificó— en lugar de diluirla en una métrica global.

7.2. Riesgos, ética y mitigaciones

El perfil de riesgo de **Seshat** es moderado: procesa conocimiento técnico interno de reuniones de ingeniería —decisiones, riesgos, tareas—, no datos personales sensibles a gran escala ni información sujeta a regulación sectorial. Esto no exime al sistema de consideraciones éticas y de seguridad, pero permite que sus mitigaciones sean proporcionadas. Los fundamentos éticos y legales se expusieron en el capítulo 2; esta sección los revisa a la luz del sistema ya implementado, centrándose en los tres riesgos de mayor incidencia práctica.

7.2.1. Alucinaciones y fiabilidad de los agentes

El riesgo técnico central de un sistema basado en LLM es la alucinación: un agente puede generar un nodo plausible pero no respaldado por la transcripción, y un nodo incorrecto en la base de conocimiento puede informar decisiones futuras. **Seshat** no confía en una sola salvaguarda, sino que superpone varias. Cada nodo queda anclado a una cita literal del texto fuente, lo que hace auditable toda afirmación frente a su origen. Los agentes admiten además una pasada opcional de auto-revisión que valida cada nodo extraído y descarta los que no superan sus propias reglas de extracción (anexo B.1.2). Y los nodos cuya confianza no alcanza el umbral calibrado se derivan a revisión humana antes de persistirse. Ninguna capa elimina el riesgo por sí sola, pero su combinación —anclaje, auto-revisión y revisión humana— lo acota a un nivel manejable para el caso de uso.

7.2.2. Privacidad de las conversaciones

Las transcripciones de reuniones pueden contener información sensible, y **Seshat** no implementa en su versión actual detección ni anonimización de información personal identificable (PII), como se anticipó en el capítulo 2. La mitigación para el MVP es organizativa antes que técnica: informar a los participantes de que la reunión se graba y de que su contenido se procesa y se estructura de forma permanente, de modo que exista consentimiento informado y una base legal para el tratamiento. Es una medida sencilla y suficiente para el despliegue interno previsto; la detección automática de PII queda como línea de trabajo futuro para escenarios de mayor exposición.

7.2.3. Dependencia de proveedores externos

Delegar la transcripción, la inferencia y los *embeddings* en proveedores externos plantea dos preocupaciones: que el contenido de las reuniones salga de la organización, y la dependencia de un tercero. La primera se mitiga por diseño. Aunque **Seshat** admite proveedores públicos como OpenAI o Anthropic, su capa de abstracción de proveedores permite igualmente Azure OpenAI y Amazon Bedrock, que pueden servir los modelos dentro de la red privada de la organización mediante *endpoints* privados, de modo que el contenido de las reuniones no abandona el perímetro de red. La segunda se mitiga por la propia intercambiabilidad: al quedar todos los proveedores tras una interfaz común, cambiar de uno a otro es una cuestión de configuración y no de refactorización (capítulo 3), lo que evita el bloqueo con un único proveedor.

8. Conclusiones y Trabajo Futuro

8.1. Conclusiones

Este trabajo se propuso diseñar e implementar un sistema capaz de extraer conocimiento estructurado de reuniones técnicas, gestionar su evolución en el tiempo mediante un grafo, e incorporar la revisión humana y la evaluación calibrada como requisitos de calidad. El resultado, **Seshat**, es un sistema completo y desplegable que cumple ese objetivo general. Las conclusiones se organizan en torno a los cinco objetivos específicos planteados en la introducción.

El **flujo multi-agente de extracción** (objetivo 1) se materializó en dos pasadas: una de identificación, con un agente especializado por tipo de concepto, y otra de resolución, que infiere las relaciones semánticas de cada nodo nuevo con el conocimiento preexistente recuperado por similitud. La inversión de la premisa habitual de la RAG —usar la recuperación no para responder preguntas, sino para construir y mantener el propio grafo— resultó ser la decisión conceptual que articula todo el sistema.

La **revisión humana asistida por confianza** (objetivo 2) se implementó como una compuerta entre ambas pasadas: cada nodo recibe una puntuación heurística que decide si se aprueba de forma automática o se deriva a un revisor, cuyo umbral se calibra —en vez de fijarse a mano— para maximizar la cobertura sin comprometer la precisión de lo aprobado. El sistema puede así operar de forma plenamente automática cuando la confianza lo permite, o concentrar la atención humana en los nodos de menor certeza.

El **marco de evaluación** (objetivo 3) es, quizá, la contribución más transferible del trabajo: cinco arneses independientes miden cada etapa no determinista del *pipeline* contra un corpus etiquetado, con *scorers* deterministas que garantizan la reproducibilidad. Su veredicto no es meramente informativo, sino que se materializa en un *gate* que impide el arranque del sistema si la calidad medida cae por debajo de los umbrales fijados (capítulo 5). En la última ejecución, las cinco etapas superan sus objetivos de calidad.

La **API REST y la interfaz web** (objetivo 4) cubren el ciclo de vida completo de un trabajo —envío, transcripción, extracción, revisión y consulta del grafo— sin exigir acceso directo a la base de datos: la API es el único punto de entrada, y la interfaz una capa de conveniencia que delega íntegramente en ella. Por último, el **despliegue contenerizado con observabilidad** (objetivo 5) integra la medición del consumo de *tokens*, la latencia por etapa y las trazas de ejecución en [MLflow](#), haciendo el rendimiento y el coste comparables entre versiones.

En conjunto, **Seshat** demuestra que un sistema basado en LLM puede alcanzar una fiabilidad apta para producción no por la infalibilidad de sus modelos, sino por la disciplina de ingeniería que los rodea: el anclaje de cada afirmación a su evidencia, la superposición de salvaguardas contra la alucinación, y —sobre todo— la conversión de la calidad en una magnitud medible, reproducible y exigible como condición de despliegue. Las limitaciones conocidas y las líneas de continuación se detallan a continuación.

8.2. Trabajo futuro

El desarrollo dejó abiertas varias líneas de continuación —incluidas las capacidades excluidas del MVP (sección 1.3)—, que se organizan aquí en cuatro categorías según la razón por la que permanecen pendientes: el endurecimiento que exigiría un despliegue de producción, las extensiones del núcleo generativo aplazadas por tiempo o por depender de datos reales, la profundización del grafo de conocimiento —trabajo de modelado de datos más que de IAG—, y la integración con sistemas externos y nuevas fuentes de entrada.

8.2.1. Endurecimiento para producción

Capacidades que un despliegue real exigiría, pero que exceden lo que un prototipo de investigación necesita demostrar.

- **Despliegue en infraestructura cloud.** El sistema corre en local contra LocalStack; llevarlo a una nube real, con su infraestructura definida como código, queda pendiente.
- **Autenticación con proveedor de identidad externo (SSO).** El modelo de claves de API basta para la base de usuarios prevista; un IdP externo —por ejemplo con JWT— sería el paso natural al crecer o al necesitar inicio de sesión único.
- **Cola de tareas durable y escalado horizontal.** El *worker* asíncrono en proceso (sección 4.5) es suficiente para el MVP, pero mantiene los trabajos en memoria: una caída del proceso los pierde y no admite más de una instancia. Una cola durable respaldada por Redis permitiría persistir los trabajos en curso, ejecutar varias réplicas del *worker* en paralelo y, sobre esa base, incorporar mecanismos como la caducidad de las revisiones pendientes que hoy no tienen plazo.
- **Reintento de trabajos consciente del progreso ya realizado.** Actualmente, reintentar un trabajo fallido recupera el archivo de entrada original desde el almacenamiento de *blobs* y repite el *pipeline* completo desde cero, incluida una nueva llamada al proveedor de transcripción cuando el origen es audio. Sin embargo, cada etapa ya persiste su artefacto durable en *blob storage*, de modo que la información para reanudar sin repetir trabajo ya existe; lo que falta es un reintento que la aproveche. La idea es sondear los artefactos en orden inverso de etapa, localizar el más avanzado que ya esté presente y reanudar desde ahí, evitando las llamadas externas redundantes y facturables.

8.2.2. Extensiones del núcleo generativo

Funcionalidades alineadas con los objetivos del sistema, pospuestas por falta de tiempo o por requerir datos de producción para validarse.

- **Arneses de evaluación para los componentes de recuperación aún sin cubrir:** el extractor de palabras clave, el generador *multi-query* y el *reranker*. Los tres son agentes ligeros de propósito único, análogos al de *grounding*, por lo que un arnés propio, es decir, un corpus etiquetado más un *scorer* determinista, encaja de forma natural en el marco existente. El más difícil de evaluar es el generador *multi-query*: al pedirle reformulaciones libres de la consulta, la salida esperada no es única, lo que sugiere explorar señales como la NLI —aplazada como señal de confianza (sección 4.4.2) pero aún sin analizar en este contexto— para juzgar si cada variante preserva la intención original.

- **Recuperación agéntica.** Un agente autónomo con herramientas de consulta como alternativa al *pipeline* RAG, que recupere nodos por razonamiento iterativo en lugar de por similitud vectorial.
- **Señal de confianza basada en NLI.** Complementar las heurísticas con una segunda señal, ortogonal, que verifique si la descripción de un nodo se deduce de su cita fuente; se aplaza por requerir un modelo local con latencia aceptable y por quedar pendiente confirmar que aporta información realmente ortogonal a las heurísticas.
- **Detección y anonimización de PII.** Identificar y ocultar información personal en las transcripciones antes de procesarlas, para escenarios de mayor exposición que el despliegue interno previsto.
- **Captura de tareas sin responsable.** La identificación descarta hoy los ítems de acción sin un responsable nombrado; capturarlos con confianza reducida se pospone hasta contar con evidencia de su necesidad en transcripciones reales.

8.2.3. Profundización del grafo de conocimiento

Trabajo centrado en el modelo de datos, su ciclo de vida y su almacenamiento, más que en los componentes de IAG.

- **Archivado de nodos (borrado lógico).** La base de conocimiento sigue hoy un modelo de sólo-adición: un nodo aprobado no puede retirarse salvo por la re-ingesta forzada, que lo elimina físicamente. Un estado **ARCHIVED** permitiría un borrado lógico reversible —excluir el nodo de la búsqueda y la navegación, y retirar su *embedding* del almacén vectorial, sin perder su historial de auditoría—, de modo que la re-ingesta forzada archivara los nodos del trabajo anterior en lugar de destruirlos.
- **Eje de ciclo de vida de los nodos.** El modelo actual distingue el tipo y el estado de un nodo (vigente, supersedido, enmendado), pero no si el asunto que representa sigue *abierto* o ya se ha *cerrado*. Un tercer eje de ciclo de vida —una pregunta abierta que pasa a **RESOLVED**, un riesgo a **MITIGATED**, una decisión a **ENACTED** cuando una relación entrante la completa— enriquecería tanto las consultas al grafo como el filtrado de candidatos en la recuperación, evitando arrastrar a la resolución nodos ya zanjados. Su despliegue está previsto por fases, e implicaría ampliar el vocabulario de relaciones (por ejemplo, una arista **implements** de un ítem de acción hacia la decisión que materializa).
- **Revisión humana de las relaciones inferidas.** Una fase de revisión para las aristas, análoga a la que hoy existe para los nodos, que hoy se escriben sin aprobación explícita.
- **Backend de grafo nativo.** La base de conocimiento se almacena hoy en PostgreSQL; un *backend* de grafo nativo como [Neo4j](#), tras la abstracción de almacén ya existente, habilitaría travesías y consultas sobre el grafo más expresivas que las que permite el modelo relacional actual.
- **Validación de integridad del grafo.** Detección de nodos duplicados, comprobación de integridad de las relaciones y guardas contra referencias circulares.
- **Umbral del *gate* por tipo de relación.** El *gate* de resolución aplica hoy un umbral uniforme por tipo de nodo origen; discriminar por tipo de relación —una vez el corpus sea lo bastante grande— permitiría exigir más a las relaciones de frontera más nítida y menos a las ambiguas, en línea con la limitación de la coincidencia exacta de terna señalada en la sección 5.5.

8.2.4. Integración y nuevas fuentes de entrada

Líneas que amplían los límites del sistema: qué formatos puede ingerir y con qué sistemas externos se conecta.

- **Entrada de vídeo y diarización de locutores.** Aceptar archivos de vídeo —con extracción de audio mediante ffmpeg— y segmentar la transcripción por locutor; ambas requieren muestras de producción para validar su calidad y un endurecimiento de las dependencias necesarias.
- **Inicialización de la base desde un corpus documental.** Sembrar la base de conocimiento a partir de documentación existente antes de la primera reunión, como herramienta complementaria de arranque. Sus fuentes naturales son las herramientas de gestión del conocimiento donde ya reside esa documentación, como [Notion](#) o [Confluence](#).

A. Guía de Despliegue y Manual de Usuario

Este anexo describe cómo levantar **Seshat** y operarlo de principio a fin. El despliegue de referencia se empaqueta como un conjunto de contenedores autocontenido, de modo que un único comando arranca la base de datos, los servicios auxiliares y la aplicación. La última sección recorre la interfaz web como manual de usuario.

A.1. Requisitos previos

Para minimizar las dependencias que recaen sobre quien lo despliega, **Seshat** se distribuye completamente dockerizado. Por eso, el único requisito para arrancar el sistema completo es **Docker** con el complemento **Compose v2** (`docker compose`); los contenedores aportan todo lo demás —PostgreSQL, el almacén de secretos y de *blobs*, el servidor de trazas y los dos servicios de Seshat—, sin instalar nada más en la máquina anfitriona. El único estado que sobrevive a los contenedores vive en volúmenes de Docker, de modo que detenerlos y volver a levantarlos no pierde ni la base de datos ni las trazas. La sección A.3 detalla cada servicio y el orden en que arrancan.

Para el desarrollo —ejecutar los tests, los arneses de evaluación o la API fuera de los contenedores— se requiere además **Python 3.13** o superior y el gestor de paquetes `uv`¹. A partir del fichero `uv.lock` versionado, `uv` reconstruye el entorno de forma determinista: instala las versiones exactas allí fijadas y resuelve hasta el modelo de spaCy que sostiene las heurísticas de puntuación de confianza (sección 4.4.2), que de otro modo exigiría una descarga aparte.

A.2. Configuración de credenciales

La configuración se suministra mediante dos ficheros de entorno que se copian de sus plantillas versionadas:

- `.env` (a partir de `.env.example`) cumple dos funciones: Docker Compose lo lee automáticamente para sustituir las variables de infraestructura del `docker-compose.yml` —credenciales de PostgreSQL, puertos, nombre del *bucket*, región—, y `python-dotenv` lo carga en las ejecuciones locales fuera de los contenedores, donde añade las claves de API y los ajustes que apuntan los servicios a `localhost`.
- `.env.docker` (a partir de `docker/.env.docker.example`) se inyecta explícitamente en los contenedores `seshat-api` y `localstack`. Contiene las claves de API —que se siembran en el almacén de secretos al arrancar— y los ajustes de configuración de Seshat que sobrescriben los valores por defecto.

¹Gestor de paquetes y proyectos de Python escrito en Rust: <https://docs.astral.sh/uv/>.

Las credenciales no se leen directamente del entorno en tiempo de ejecución, sino a través de un **resolutor de secretos** seleccionable por configuración. Con el proveedor `env` se leen de las variables de entorno, adecuado para ejecuciones locales; con el proveedor `aws` se recuperan de AWS Secrets Manager. El despliegue en contenedores usa este último apuntando a **LocalStack**, que emula el servicio: así el arranque local ejercita exactamente el mismo camino de código que un despliegue en la nube, sin depender de infraestructura real.

Las claves mínimas para un arranque funcional son la clave raíz que gobierna la administración (`SESHAT_ROOT_API_KEY`) y las de los proveedores del pipeline por defecto: `OPENAI_API_KEY` (*embeddings* y *grounding*), `ANTHROPIC_API_KEY` (identificación y resolución) y `ASSEMBLYAI_API_KEY` (transcripción).

A.3. Arranque con Docker Compose

Una vez copiados y rellenados los dos ficheros de entorno, el conjunto completo de contenedores se levanta desde la raíz del proyecto con un único comando:

```
docker compose up
```

En la primera ejecución, Docker Compose construye las imágenes de Seshat que aún no existen² y arranca los seis servicios de la tabla A.1 en el orden que imponen sus dependencias. El servicio `migrate` aplica las migraciones de Alembic (`alembic upgrade head`), que crean los esquemas `ops` y `knowledge_base` descritos en la sección B.4, y termina; `seshat-api` sólo arranca una vez que `migrate` ha concluido con éxito y que PostgreSQL, LocalStack y MLflow están sanos, y `seshat-ui` espera a su vez a que la API lo esté.

Servicio	Origen	Puerto	Función
<code>postgres</code>	<code>pgvector/pgvector:pg17</code>	5432	Base de datos: esquemas <code>ops</code> y <code>knowledge_base</code> más los <i>embeddings</i> de <code>pgvector</code> .
<code>migrate</code>	Dockerfile (<code>migrate</code>)	—	Aplica las migraciones de Alembic una sola vez y termina.
<code>mlflow</code>	<code>mlflow:v3.12.0</code>	5000	Servidor de trazas y métricas de observabilidad.
<code>localstack</code>	<code>localstack:3</code>	4566	Emula AWS Secrets Manager y S3; siembra los secretos y crea el <i>bucket</i> al arrancar.
<code>seshat-api</code>	Dockerfile (<code>api</code>)	8000	API REST y cola de trabajos en proceso.
<code>seshat-ui</code>	Dockerfile (<code>ui</code>)	8501	Interfaz web (Streamlit) sobre la API.

Tabla A.1.: Servicios de `docker compose` y el puerto que cada uno expone en la máquina anfitriona.

²Compose sólo construye las imágenes ausentes; tras modificar el código fuente hay que forzar la reconstrucción con `docker compose up --build`, o construir las imágenes manualmente con `docker compose build <service_name>`. Alternativamente, se puede crear un fichero `docker-compose.override.yml` y definir en él los ajustes personalizados, tales como volúmenes a las carpetas donde se encuentra el código fuente.

El arranque de la API atraviesa además dos comprobaciones que actúan como salvaguardas: un sondeo a todos los proveedores de modelos configurados y la **verificación de la puerta de evaluación**, que aborta el arranque si no existe un `eval_gate.json` con veredicto favorable (véase el capítulo 5).

En el despliegue en local ambas comprobaciones se desactivan mediante las variables de entorno `API__SKIP_EXTERNAL_PROVIDER_PING` y `API__SKIP_EVAL_GATE`, de modo que la demostración arranca sin exigir una ejecución previa de los arneses ni que todos los proveedores estén disponibles. En un despliegue real ambas se dejan activas: se ejecuta antes `seshat eval` para producir una puerta favorable y sólo entonces se permite servir tráfico.

Una comprobación adicional, ajena al *gate* pero también resuelta por configuración, es la verificación TLS de las llamadas salientes a proveedores externos. En una red corporativa cuyo proxy inyecta un certificado raíz propio, ese certificado suele estar provisionado en el almacén de confianza del sistema operativo pero no en el que Python empaqueta por defecto, lo que hace fallar la verificación aun cuando el tráfico es legítimo. La variable de entorno `USE_OS_TRUSTSTORE` resuelve esto sin desactivar la verificación: activada, hace que el módulo `ssl` de Python delegue en el almacén de confianza del sistema operativo en lugar del suyo propio, de modo que las llamadas a través de ese proxy quedan verificadas igual que las demás.

Una vez sanos todos los servicios, la interfaz web queda accesible en `localhost:8501`, la API en `localhost:8000` y la interfaz de MLflow en `localhost:5000`. Fuera de los contenedores, los mismos dos pasos se ejecutan a mano con la línea de comandos: `seshat migrate` aplica las migraciones y `seshat api` levanta el servidor.

A.4. Manual de usuario

Los usuarios operan **Seshat** a través de una interfaz web: el cliente fino de Streamlit cuyo diseño se motiva en la sección 6.3. Se apoya en la misma API REST documentada en el anexo B.5, de modo que cualquier acción de la interfaz tiene su equivalente en un *endpoint*. Se organiza en cuatro pantallas: **Jobs** envía grabaciones o transcripciones y gobierna su revisión hasta que se escriben en el grafo; **Graph** navega, busca y explora la base de conocimiento, incluida la travesía de impacto de un nodo; **Manual Actions** cubre las operaciones ajenas al pipeline —crear nodos y relaciones a mano, anular contenido o forzar la reingesta de un trabajo—; y el **Admin Panel**, reservado a la clave raíz, provisiona las claves de API y sus roles.

Dos guías de usuario, mantenidas junto al código para que evolucionen con él, cubren el detalle de cada superficie. El recorrido completo de la interfaz, con un escenario de ejemplo de principio a fin, está en [docs/user-guide.md](#); la operación de los arneses de evaluación —ejecutarlos, leer los resultados en MLflow y calibrar los umbrales— se recoge en [docs/eval-user-guide.md](#).

B. Detalles Técnicos

B.1. Arquitectura de agentes

Seshat llama «agente» a todo componente que envuelve una invocación estructurada a un LLM con reintentos y control de concurrencia. Todos comparten una clase base abstracta, `_BaseAgent`, y desde ahí se distinguen sólo dos formas de especialización: la primera son las familias parametrizadas del *pipeline* de extracción —los agentes de identificación y los de resolución—, y la segunda son agentes de propósito único y diverso, todos descritos a continuación.

Por un lado, las dos **familias parametrizadas** interponen una base intermedia para generar, por configuración, un agente concreto por tipo de concepto o par de tipos; cada familia admite además una variante **reflectiva** opcional que añade una segunda pasada de auto-revisión sobre la salida del agente primario. Por otro, los **agentes de propósito único** extienden `_BaseAgent` directamente, y sólo difieren entre sí en su prompt y su esquema de salida: el de *grounding* verifica el anclaje de un nodo en su cita y el de agrupación consolida los conceptos fragmentados, mientras que el extractor de palabras clave y el generador *multi-query* sirven a la recuperación de información de la base de datos vectorial.

Cada familia se expone tras un *registro* que actúa como fachada: lee la configuración, instancia los agentes concretos habilitados y los ofrece al orquestador como una colección uniforme, de modo que este dialoga con el registro y no con cada agente por separado. Hay uno por familia —`IdentificationRegistry` y `ResolutionRegistry`, este último apoyado a su vez en un registro de mismo tipo y otro de tipo cruzado—. Gracias a esta abstracción, activar o desactivar un tipo de concepto es un cambio de configuración y no del código del orquestador. Por otro lado, los agentes de propósito único, al no parametrizarse, no pasan por ningún registro y se construyen directamente.

B.1.1. Jerarquía de clases y registro

La jerarquía completa de clases, colgando toda de `_BaseAgent`, es la siguiente:

- **Familia de identificación:** base `_BaseIdentificationAgent`, que extiende `_BaseAgent` con un esquema de salida genérico parametrizado por el tipo de concepto:
 - `DecisionIdentificationAgent`
 - `RiskIdentificationAgent`
 - `ActionItemIdentificationAgent`
 - `OpenQuestionIdentificationAgent`

Además, `ReflectiveIdentificationAgent` envuelve a cualquiera de estos agentes para añadir una segunda pasada de auto-revisión sobre su salida.

- **Familia de resolución:** base `_BaseResolutionAgent`, con dos sub-bases según si el nodo fuente y el destino son del mismo tipo de concepto o de tipos distintos. Por un lado, `BaseSameTypeResolutionAgent` relaciona nodos de un mismo tipo y parametriza los tipos de relación válidos para ese `ConceptType` con un agente concreto por concepto:

- `DecisionResolutionAgent`
- `RiskResolutionAgent`
- `ActionItemResolutionAgent`
- `OpenQuestionResolutionAgent`

Por el otro, `BaseCrossTypeResolutionAgent` relaciona nodos de tipos distintos, con un agente concreto por cada tipo fuente que fija los pares (`source_type`, `target_type`) que ese agente gestiona:

- `DecisionCrossTypeResolutionAgent`
- `RiskCrossTypeResolutionAgent`
- `ActionItemCrossTypeResolutionAgent`
- `OpenQuestionCrossTypeResolutionAgent`

Análogamente, `ReflectiveResolutionAgent` envuelve a cualquiera de estos agentes para arbitrar los ambiguos en una segunda pasada.

- **Agentes de propósito único:** extienden `_BaseAgent` directamente, sin base intermedia:
 - `GroundingAgent`: verificación del anclaje de un nodo en su cita.
 - `GroupingAgent`: agrupación de conceptos fragmentados.
 - `KeywordAgent`: extracción de palabras clave para la búsqueda dispersa.
 - `MultiQueryAgent`: expansión multi-consulta para la búsqueda densa.

Sea cual sea su función, todos los agentes anteriores cumplen el mismo contrato heredado de `_BaseAgent`: aportan un *prompt* de sistema propio, producen una salida estructurada conforme a un modelo Pydantic dado —cuyas violaciones de tipo se rechazan y reintentan— y encapsulan sus llamadas al proveedor con reintentos de *backoff* exponencial y *jitter*. Es este contrato común, ya mencionado en el capítulo 4, el que permite tratarlos de manera uniforme pese a la diversidad de sus propósitos.

B.1.2. Agentes reflectivos

A pesar de todas las técnicas de diseño de prompts usadas (descritas en la sección B.1.4) y de forzar la salida estructurada los agentes, hay veces que estos agentes "shallow" no son suficientes. Por ello, se ha implementado un modo de **auto-revisión** (*self-review*) opcional sobre las familias de agentes parametrizadas, que añade una segunda pasada de LLM sobre la salida del agente primario.

Por una parte, el `ReflectiveIdentificationAgent` implementa un ciclo *extraer* → *validar* → *filtrar*: tras la extracción, una llamada de validación comprueba el cumplimiento lógico y semántico de cada nodo (¿satisface las reglas de extracción? ¿coincide la descripción con la cita?); los nodos que fallan se descartan. En caso de fallo de la validación, todos los nodos se devuelven sin filtrar, garantizando que el modo reflectivo nunca sea más perjudicial que el modo plano.

Por otra, el `ReflectiveResolutionAgent` actúa como árbitro para los casos genuinamente ambiguos. Sólo las entradas con `alt_rel_type` no nulo se envían a una llamada de desempate; las entradas claras se omiten por completo, lo que minimiza el coste adicional. Ante cualquier fallo del árbitro, se conserva el `rel_type` original.

Ambos agentes utilizan el patrón *subclass-and-delegate*: heredan de la misma base que los agentes planos y delegan todas las propiedades abstractas al agente interno. El orquestador y los registros de agentes no requieren ningún cambio para soportar el modo reflectivo.

B.1.3. Matriz de relaciones por par de tipos

La tabla B.1 recoge las relaciones que un agente de resolución puede asignar para cada par dirigido (tipo fuente $\rightarrow$ tipo destino): de las 12 combinaciones dirigidas entre tipos distintos, sólo 9 se habilitan.

Destino Fuente	decision	risk	action_item	open_question
decision	supersedes, amends, conflicts_with	mitigates	blocks	resolves
risk	blocks	amends	blocks	blocks
action_item	—	mitigates	supersedes, amends, conflicts_with, blocks, depends_on	—
open_question	blocks	—	blocks	amends, depends_on

Tabla B.1.: Relaciones permitidas por par de tipos (fuente $\rightarrow$ destino). La diagonal es resolución de mismo tipo; el resto, de tipo cruzado. Un guión (—) indica que ningún agente gestiona ese par en la versión actual.

B.1.4. Técnicas de diseño de prompts

El diseño de los *prompts* de los agentes combina varias técnicas complementarias, aplicadas en mayor o menor medida según la tarea de cada uno:

- **Razonamiento en cadena** (*chain-of-thought* [7]): el prompt descompone la tarea en pasos intermedios secuenciales y verificables —localizar el fragmento relevante, evaluar las compuertas lógicas, clasificar el tipo de concepto— antes de emitir la salida final. Hacer explícito el razonamiento reduce los errores al exponer los pasos intermedios a la propia lógica del modelo.
- **Anclaje en cita literal** (*quote-first grounding*): antes de describir un nodo, el agente debe localizar y copiar el fragmento literal del texto fuente que lo respalda. Esto reduce la alucinación al obligar al modelo a anclar cada afirmación en evidencia concreta antes de sintetizarla; es central en la identificación y en el propio agente de *grounding*, cuya tarea es verificar ese anclaje.

- **Compuertas lógicas:** el prompt enumera condiciones necesarias y suficientes para emitir un nodo o una relación —por ejemplo, para que un **risk** sea válido deben cumplirse simultáneamente cuatro condiciones: modo de fallo concreto, implicación sustantiva del grupo, no totalmente resuelto en el mismo intercambio, y no un requisito sin consecuencia declarada. Si alguna condición no se satisface, no debe emitirse. Es la técnica más presente en los agentes de resolución, cuyas fronteras entre tipos de relación son sutiles.
- **Contraejemplos etiquetados:** el prompt incluye ejemplos de casos límite que *no* deben producirse, con explicación de por qué. Esto delimita la frontera semántica entre categorías próximas —por ejemplo, **risk** frente a **open_question** en identificación, o **amends** frente a **supersedes** en resolución.
- **Delimitadores XML:** en los agentes que reciben la transcripción —identificación y *grounding*—, el contenido de la reunión y los *hints* de la base de conocimiento se encierran en bloques `<transcript>` y `<kb_hint>`. Las instrucciones del sistema indican explícitamente que cualquier texto con apariencia de instrucción dentro de esos bloques debe tratarse como datos y no ejecutarse, lo que mitiga la inyección de prompts desde el contenido de la reunión.
- **Salida estructurada forzada:** común a todos los agentes. Los esquemas Pydantic se inyectan como instrucción de herramienta en la llamada al modelo; las violaciones de tipo activan reintentos automáticos, lo que elimina la ambigüedad de formato y hace el post-procesamiento determinista.

B.1.5. Caché de prompts y optimización de latencia

La caché de *prompts* a nivel de proveedor opera sobre el *prefijo* de la petición: el tramo inicial e idéntico de una llamada a otra, que en estos agentes es el prompt de sistema. La primera llamada que envía ese prefijo paga una prima de escritura y deja el prefijo en la caché del proveedor durante una ventana temporal corta; cualquier llamada posterior que reutilice un prefijo ya visto obtiene descuento de lectura sobre esos tokens. El mecanismo se activa de forma distinta según el proveedor —en Anthropic hay que marcar el prefijo con la cabecera `cache_control`; OpenAI cachea automáticamente los prefijos compartidos—, pero la lógica de cuándo conviene es la misma: sólo compensa cuando ese prefijo se reutiliza en varias llamadas *sucesivas*, no concurrentes, dentro de un mismo trabajo.

Esa condición no la cumplen todos los agentes por igual, porque las llamadas al LLM se lanzan en paralelo bajo un límite de concurrencia; es decir, en oleadas. La primera oleada falla la caché —todas sus llamadas escriben a la vez, sin lecturas previas—, y sólo las oleadas posteriores, o los reintentos, la aprovechan. De ahí que la caché rinda cuando el número de llamadas supera el límite de concurrencia:

- En la **resolución**, donde cada nodo nuevo se compara con muchos candidatos, el prompt de sistema —y el contexto de nodos hermanos— se reutiliza a lo largo de sucesivas oleadas.
- En las **variantes reflectivas**, la segunda pasada de auto-revisión reemite el mismo prompt de sistema inmediatamente después de la primera, garantizando una lectura de caché.
- En el **grounding**, sólo cuando hay más nodos que verificar que el límite de concurrencia; en caso contrario, la única oleada no llega a beneficiarse.

La **identificación** es el caso límite: sus agentes de concepto se lanzan todos a la vez —sin límite de concurrencia— y cada uno usa un prompt de sistema distinto, de modo que dentro de un trabajo no hay ningún prefijo que se reutilice y amortice la escritura. Por eso no cachea su prompt salvo en modo reflectivo, donde la pasada de validación sí lo reemite y la caché rinde. Su transcripción, además, se excluye siempre de la caché por la misma razón de fondo: al lanzarse todos los agentes en paralelo, cualquier lectura fallaría.

B.2. Capa de observabilidad

La medición del consumo y la latencia descrita en la sección 4.7 se apoya en dos rastreadores —`UsageTracker` y `LatencyTracker`— que se instalan como *callbacks* de `LangChain`. El reto de implementación es que los agentes se ejecutan de forma concurrente: la solución evita propagar el rastreador por la firma de cada método y lo publica en una variable de contexto (`contextvars.ContextVar`). Las tareas hijas heredan el valor de la variable al crearse, de modo que todos los agentes lanzados dentro de una etapa acumulan en el mismo rastreador sin cambio alguno en sus firmas.

Cada etapa del *pipeline* se envuelve con dos decoradores —`track_token_budget` y `track_latency_profile`— que, al entrar, crean el rastreador y lo fijan en la variable de contexto y, al salir, registran los totales como métricas de MLflow (tokens por tipo, y latencia mínima, media, mediana, p95 y máxima). El decorador de presupuesto lee sus límites de la configuración en tiempo de llamada, avisa al alcanzar el 90 % del cupo y aborta la etapa al superarlo en más de un 10 %; este margen tolera que varios agentes concurrentes rebasen el cupo antes de que cualquiera de ellos pueda observarlo. Los totales de cada etapa se agregan además en un rastreador a nivel de trabajo.

Dos fuentes de consumo no pasan por las llamadas al LLM y se contabilizan mediante envoltorios transparentes: `TrackingEmbeddings` envuelve el modelo de *embeddings* y cuenta sus tokens de entrada, y `TrackingTranscriber` envuelve al transcriptor y contabiliza los segundos de audio procesados. Ambos empujan su medición al rastreador activo en la variable de contexto, integrándose en el mismo presupuesto que las llamadas al LLM.

B.3. Arquitectura de evaluación

B.3.1. Estructura común de los arneses y el emparejador de identificación

Los cinco arneses comparten el mismo esqueleto de ejecución, condicionado por un detalle del marco de evaluación: `mlflow.genai.evaluate` invoca su función de predicción de forma síncrona, mientras que los componentes de **Seshat** —orquestador y agentes— son asíncronos. Para salvar esa incompatibilidad, cada arnés pre-computa todas las predicciones antes de entregar el control a MLflow: un método `_run_all_predictions` recorre el corpus, ejecuta el componente real de cada caso y almacena el resultado en una caché en memoria; la función de predicción que ve MLflow es entonces una mera consulta a esa caché. Sobre las predicciones cacheadas, MLflow aplica el *scorer* del arnés —una función determinista, sin llamadas al LLM—, agrega las métricas por caso y el arnés vuelca el veredicto al fichero de *gate* mediante `upsert_gate`, que actualiza sólo el bloque del arnés en ejecución y conserva el resto.

El emparejador de identificación merece una descripción aparte, pues es la pieza no trivial de esa cadena: debe decidir qué nodo extraído corresponde a qué nodo esperado antes de poder contar aciertos y fallos. Resuelve un emparejamiento bipartito *greedy* restringido a pares del mismo tipo de concepto. Para cada par candidato calcula una puntuación de solapamiento en $[0, 1]$ por una de dos vías: si el nodo predicho lleva cita (`quote_anchors`), la puntuación es la similitud difusa (`rapidfuzz`, *partial ratio*) entre la cita esperada y el fragmento de transcripción que abarca la predicha; si no la lleva, se recurre a una similitud difusa ponderada sobre título y descripción. Los pares se ordenan de mayor a menor puntuación y se van reclamando de forma *greedy* —cada nodo esperado y cada predicho se asignan una sola vez—; un par sólo se acepta como acierto si supera el umbral `QUOTE_MATCH_THRESHOLD` de 0.70. Los nodos predichos sin pareja cuentan como falsos positivos y los esperados sin pareja, como falsos negativos. Sobre los pares aceptados, el *scorer* puntúa además la exactitud de los campos estructurados propios de cada tipo (difusa para `assignee`, `due`, `rationale` y `context`; exacta para el subtipo de `risk`; solapamiento de conjuntos para `alternatives_considered`), señales que se registran en MLflow pero no condicionan el *gate*.

Cada arnés registra en la traza de MLflow, junto a la salida que consume el *scorer*, una vista de comparación legible: los nodos —o relaciones— esperados y predichos uno al lado del otro. El objetivo es la depurabilidad: en la interfaz de MLflow cada caso es una fila, y un caso fallido debe poder diagnosticarse desde esa fila sin abrir el fichero YAML del corpus. Un post-procesador de trazas recorta además los campos más verbosos de los nodos —citas en bruto, descripciones largas— y conserva sólo tipo, título, descripción y confianza, de modo que la comparación quepa de un vistazo.

B.3.2. Caché de evaluación y meta-scorers de calibración

Caché de predicciones. La caché de predicciones que sostiene la ejecución de los arneses es también la que permite calibrar umbrales sin repetir llamadas al LLM. Su pieza central es `read_or_run`: dada la ruta de un fichero de caché y la corrutina que produciría el resultado, devuelve el contenido cacheado si el fichero existe —cerrando la corrutina sin ejecutarla— y, en caso contrario, ejecuta la corrutina y persiste el resultado como JSON. El nombre del fichero se deriva de tres componentes: el identificador del caso de corpus, una huella (*fingerprint*) del agente y un *hash* de la entrada. La huella del agente incorpora su *prompt* y configuración, de modo que cualquier cambio en el agente invalida automáticamente sus entradas de caché sin afectar a las de otras variantes. Al terminar, cada arnés no borra la caché, sino que aplica un barrido de marcado (`sweep_stale_entries`) que elimina únicamente las entradas en ámbito que no se tocaron en la ejecución —predicciones obsoletas por un cambio de *prompt* o de entrada—, preservando las válidas para ejecuciones posteriores.

Meta-scorers de calibración. Sobre esa caché operan los dos meta-*scorers*¹ de calibración. Ambos siguen un diseño en dos fases: primero pueblan la caché con el resultado del *pipeline* para cada caso —reutilizando las predicciones que el arnés correspondiente ya hubiera generado—, y después barren el umbral sobre esos resultados en memoria, sin más entrada/salida ni llamadas al modelo.

¹Un meta-*scorer* opera un nivel por encima de un *scorer*: mientras que un *scorer* mide la calidad a un umbral *fijo*, un meta-*scorer* recorre valores discretos del rango de validez del umbral para *elegir* su mejor valor.

El *meta-scorer* de identificación barre el umbral sobre la confianza heurística en el intervalo $[0, 1]$ con paso 0.01. En cada punto reproduce qué nodos se auto-aprobarían y, aplicando el emparejador de identificación contra la verdad de referencia, calcula dos magnitudes, de forma agregada y por tipo de concepto: la **precisión** entre los nodos aprobados y la **cobertura** (qué fracción de los nodos esperados se llegó a auto-aprobar; es el *recall*, renombrado para subrayar que un nodo por debajo del umbral no se pierde, sólo se difiere a revisión humana). El umbral sugerido es el que maximiza la cobertura sujeto a que la precisión alcance un objetivo configurable (0.95 por defecto); si ninguno lo logra, se conserva el de mayor precisión, con los empates resueltos hacia el umbral más bajo. No se emplea F1 porque contaría cada nodo diferido como un falso negativo, justo la lectura que el renombrado de la cobertura descarta.

El *meta-scorer* de *retrieval* barre el umbral de similitud con paso 0.005. Para que un único resultado cacheado sirva a cualquier corte, las búsquedas se siembran sin filtro de puntuación y el umbral se aplica en memoria durante el barrido. El umbral sugerido es el que maximiza el F_2^2 macro-promediado: los casos positivos aportan su F2 —que pondera el *recall* más que la precisión— y los negativos aportan especificidad (1 si no se devuelve nada por encima del umbral, 0 en caso contrario), lo que impide que un umbral en cero, que lo aceptaría todo, domine la selección.

B.4. Esquema de la base de datos

El estado del sistema se reparte en dos esquemas de PostgreSQL con responsabilidades separadas: **ops**, que guarda el estado operativo (trabajos y claves de API), y **knowledge_base**, que guarda el grafo de conocimiento (nodos y relaciones). Los *embeddings* viven en una tercera tabla, gestionada por LangChain, que reside en el mismo PostgreSQL a través de la extensión pgvector.

Esquema ops. La tabla **jobs** registra el ciclo de vida de cada *pipeline* —su estado, marcas de tiempo, el *blob* de entrada y el identificador de la ejecución de MLflow— e incluye una clave de idempotencia única (U) que sostiene el control de duplicados. La tabla **api_keys** guarda las credenciales como *hashes* junto con su rol. Ambas son independientes entre sí, como muestra la figura B.1.

Esquema knowledge_base. Aquí reside el grafo (figura B.2). **kb_nodes** almacena cada nodo con su contenido, su cita de anclaje y sus metadatos; **kb_relationships** es la tabla de aristas, y su carácter de grafo se ve en que **source_id** y **target_id** son ambos claves foráneas hacia **kb_nodes.node_id**. Cada arista tiene una clave primaria subrogada **rel_id** y una restricción de unicidad sobre la terna (**source_id**, **target_id**, **rel_type**) —marcada U1 en la figura— que impide duplicar una misma relación.

Almacén vectorial. Los *embeddings* de los nodos residen en la tabla **langchain_pg_embedding**, que LangChain materializa de forma diferida en la primera conexión. Sobre ella se añade una columna generada de *tsvector* con un índice *GIN* que habilita la búsqueda por palabras clave; la búsqueda semántica usa el índice vectorial de pgvector sobre la columna de *embeddings*. El vínculo con el grafo es el **node_id**, que se conserva en los metadatos de cada *embedding* para poder recuperar el nodo correspondiente.

² F_β es la media armónica ponderada de precisión (P) y *recall* (R); con $\beta = 2$ se obtiene $F_2 = \frac{5PR}{4P+R}$.

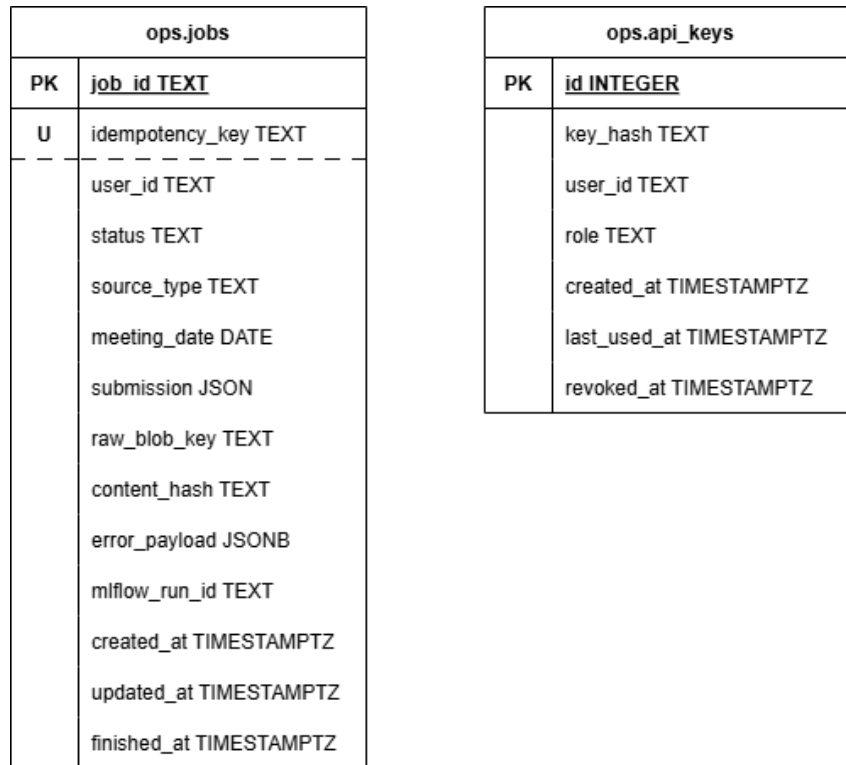


Figura B.1.: Esquema ops. Las dos tablas no comparten claves foráneas; U marca la clave de idempotencia única de jobs.

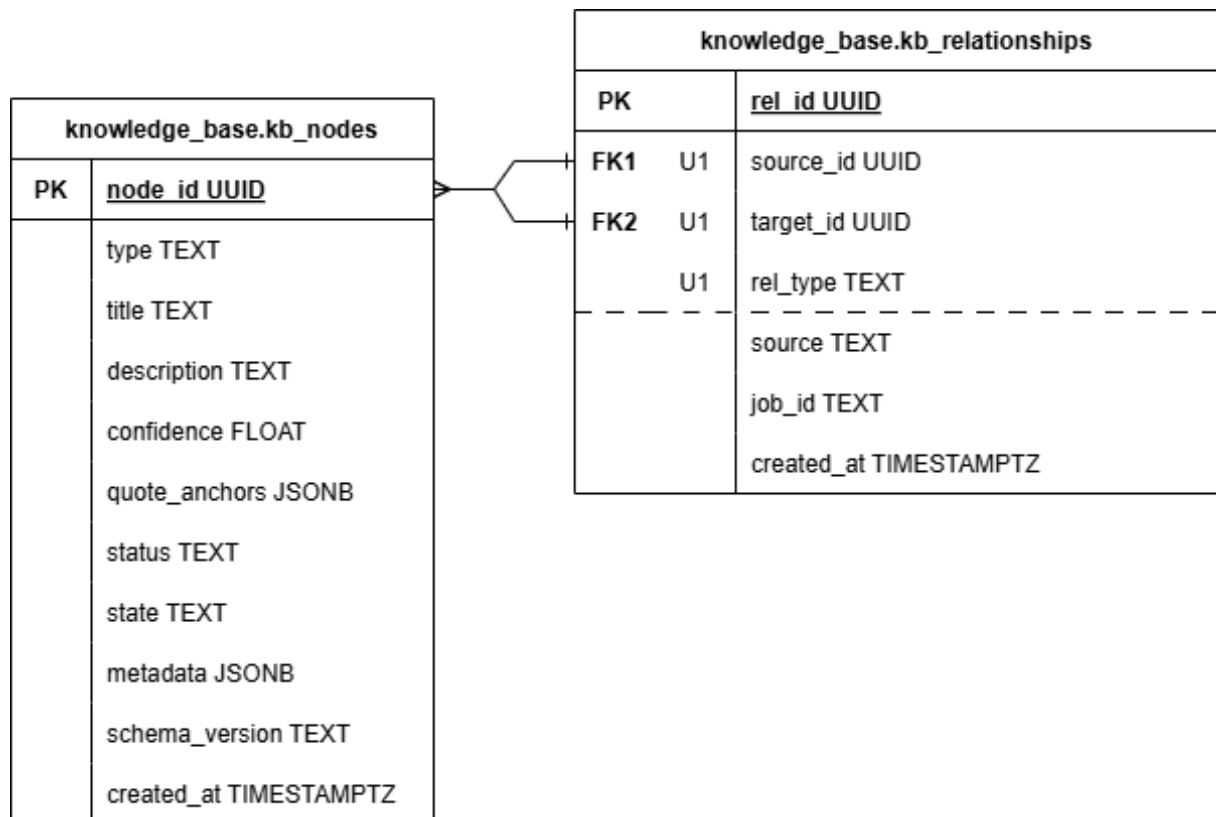


Figura B.2.: Esquema knowledge_base. PK marca la clave primaria; FK1 y FK2, las dos claves foráneas de kb_relationships hacia kb_nodes; y U1, las columnas de la restricción de unicidad de la arista.

B.5. Referencia de la API REST

La tabla B.2 agrupa los *endpoints* por router: salvo `/admin` —autenticado con la clave raíz— y `health`, todos exigen una clave de API válida y el rol indicado en la sección 4.5.

Método	Ruta	Descripción
/jobs — ciclo de vida del <i>pipeline</i>		
POST	/jobs	Crear un <i>pipeline</i> a partir de un archivo de entrada
GET	/jobs	Listar <i>pipelines</i> con filtros
GET	/jobs/{id}	Consultar el estado de un <i>pipeline</i>
GET	/jobs/{id}/results	Recuperar los nodos extraídos
GET	/jobs/{id}/transcript/excerpt	Extraer un fragmento de la transcripción
POST	/jobs/{id}/approve	Enviar decisiones de revisión y disparar la escritura
POST	/jobs/{id}/retry	Reintentar un <i>pipeline</i> fallido
/graph — consulta de la base de conocimiento		
GET	/graph	Consultar nodos por filtros
GET	/graph/search	Búsqueda semántica sobre los nodos
GET	/graph/{id}	Obtener un nodo por su identificador
GET	/graph/{id}/neighbours	Vecinos directos (profundidad 1)
GET	/graph/{id}/detail	Nodo junto con sus vecinos directos
GET	/graph/{id}/impact	Travesía multi-salto de impacto
POST	/graph/nodes	Añadir un nodo manualmente
POST	/graph/nodes/bulk	Añadir nodos en lote
POST	/graph/nodes/resolve	Resolver relaciones para nodos dados
PUT	/graph/nodes/{id}	Editar un nodo
PUT	/graph/nodes/{id}/override	Anular contenido con justificación
DELETE	/graph/nodes/{id}	Eliminar un nodo
DELETE	/graph/nodes/bulk	Eliminar nodos en lote
GET	/graph/relationships	Listar relaciones
POST	/graph/relationships	Crear una relación
DELETE	/graph/relationships/{id}	Eliminar una relación
/admin — provisión de claves (clave raíz)		
GET	/admin/api-keys	Listar claves de API
POST	/admin/api-keys	Crear una clave de API
DELETE	/admin/api-keys/{id}	Revocar una clave de API
Identidad y salud		
GET	/me	Resolver la clave de API a usuario y rol
GET	/health	Sondeo de salud de la API
GET	/health/components	Estado de las dependencias externas

Tabla B.2.: *Endpoints* de la API que expone **Seshat**, agrupados por router. Todos cuelgan del prefijo de versión `/v1`.

B.6. Configuración completa de referencia

La referencia completa de parámetros de configuración y sus variables de entorno se mantiene junto al código, en [docs/configuration.md](#), para que evolucione con él sin quedar desactualizada en este documento.

B.7. *Prompts* de los agentes

Por la misma razón, los *prompts* de sistema de los agentes no se reproducen en este documento: cada uno vive junto al agente que lo usa, en su propio script, y todos los agentes están centralizados en [src/seshat/app/agents/](#).

Índice de figuras

3.1. Arquitectura de Seshat : la interfaz envía trabajos a la API por HTTP; la API los encola para el Worker; el Worker ejecuta el procesamiento y persiste los resultados en la capa de almacenamiento.	9
3.2. Ejemplo de grafo de conocimiento en Seshat . El nodo con borde discontinuo representa una decisión previa en estado SUPERSEDED ; las relaciones entre nodos de distinto tipo se muestran con flechas de línea discontinua.	11
3.3. Ciclo de vida de un <i>pipeline</i> en Seshat . IDENTIFYING y RESOLVING corresponden a las dos pasadas de extracción; el paso <i>HITL</i> indica la revisión humana opcional; y en WRITING se persisten los nodos y sus relaciones en la base de conocimiento. Las flechas discontinuas rojas señalan transiciones a error y la flecha inferior muestra el reintento mediante llamada a la API.	13
4.1. Estructura de paquetes de Seshat	14
4.2. Flujo de la primera pasada de extracción. La transcripción normalizada en la ingesta alimenta el <i>fan-out</i> de agentes de identificación —el de decisiones incluye además el paso de agrupación—; sus salidas se deduplican, se puntúan por confianza —heurísticas y, opcionalmente, verificación por <i>grounding</i> — y reciben un estado de aprobación (pasos descritos en la sección 4.4.2) antes de persistirse como nodos nuevos.	16
4.3. Flujo de recuperación para un nodo fuente, en su configuración completa (modo HYBRID). El SearchEngine combina una pata semántica (búsqueda densa, con expansión <i>multi-query</i> opcional) y una pata dispersa por palabras clave, fundidas mediante RRF. El NodeRetriever obtiene los nodos de la base de conocimiento, los expande con sus vecinos directos y opcionalmente aplica un reranking (recuadro discontinuo) antes de devolver los nodos objetivo.	18
4.4. Flujo de la segunda pasada. Para cada nodo fuente de la primera pasada se recuperan sus nodos objetivo (recuperación descrita en la sección 4.4.3); el par fuente-objetivos se envía en paralelo a las familias de resolución de mismo tipo (un agente por tipo) y de tipo cruzado (hasta nueve pares activos). Un paso de validación mediante reglas heurísticas filtra las relaciones inferidas antes de persistirlas.	19
5.1. Barrido del umbral de confianza sobre el corpus de identificación. La cobertura (fracción de nodos auto-aprobados) decrece al subir el umbral, mientras la precisión sobre los auto-aprobados crece. El punto $t^* = 0,76$ es el de mayor cobertura entre los que alcanzan la precisión objetivo $p_{obj} = 0,95$	32
B.1. Esquema ops . Las dos tablas no comparten claves foráneas; U marca la clave de idempotencia única de jobs	52

B.2. Esquema <code>knowledge_base</code> . PK marca la clave primaria; FK1 y FK2, las dos claves foráneas de <code>kb_relationships</code> hacia <code>kb_nodes</code> ; y U1, las columnas de la restricción de unicidad de la arista.	52
---	----

Índice de cuadros

3.1.	Tipos de nodo en el grafo de conocimiento de Seshat	12
3.2.	Tipos de relación en el grafo de conocimiento de Seshat	12
5.1.	Criterios del <i>gate</i> de evaluación: métricas medidas por cada arnés, umbral de aprobado y si su incumplimiento impide el arranque del sistema. Los umbrales de identificación varían según el tipo de concepto; se indica su rango y los valores exactos por tipo se recogen en la tabla 5.3.	25
5.2.	Composición del corpus de evaluación por arnés y <i>tier</i> , con el rol que cumple cada <i>tier</i> y su número de casos.	26
5.3.	Precisión, recall y tasa espuria de identificación por tipo de concepto. Cada valor está promediado sobre los casos del corpus.	27
5.4.	Resultados del arnés de agrupación. Cada valor está promediado sobre los casos del corpus.	28
5.5.	Resultados del arnés de <i>grounding</i> . Ambos valores se agregan sobre todos los nodos del corpus en una única matriz de confusión.	28
5.6.	Resultados del arnés de recuperación. Cada valor está promediado sobre los casos del corpus.	29
5.7.	Precisión y recall de resolución por tipo de concepto origen. Cada valor está promediado sobre los casos del corpus.	30
5.8.	Precisión y recall de resolución por tipo de concepto origen y familia de agente, puntuados a nivel de relación individual según los tipos de concepto de su nodo origen y destino.	30
A.1.	Servicios de docker compose y el puerto que cada uno expone en la máquina anfitriona.	43
B.1.	Relaciones permitidas por par de tipos (fuente → destino). La diagonal es resolución de mismo tipo; el resto, de tipo cruzado. Un guión (—) indica que ningún agente gestiona ese par en la versión actual.	47
B.2.	<i>Endpoints</i> de la API que expone Seshat , agrupados por router. Todos cuelgan del prefijo de versión /v1	53

Acrónimos

ASR	Automatic Speech Recognition
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
NLI	Natural Language Inference
RRF	Reciprocal Rank Fusion
OTel	OpenTelemetry
API	Application Programming Interface
HITL	Human-in-the-Loop
TFM	Trabajo de Fin de Máster
MVP	Minimum Viable Product
CI	Continuous Integration
IAG	Inteligencia Artificial Generativa
IaC	Infrastructure as Code

Bibliografía

- [1] R. H. Wilkinson, *The Complete Gods and Goddesses of Ancient Egypt*. London: Thames & Hudson, 2003, ISBN: 978-0-500-05120-7.
- [2] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., 2017.
- [3] T. B. Brown et al., «Language Models are Few-Shot Learners,» *Advances in Neural Information Processing Systems*, vol. 33, págs. 1877-1901, 2020.
- [4] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» en *Advances in Neural Information Processing Systems*, vol. 33, 2020, págs. 9459-9474.
- [5] M. Zaharia et al., «Accelerating the Machine Learning Lifecycle with MLflow,» en *Workshop on ML Systems at NeurIPS*, 2018.
- [6] S. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023. arXiv: [2210.03629 \[cs.CL\]](#).
- [7] J. Wei et al., «Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,» en *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [8] Y. Gao et al., *Retrieval-Augmented Generation for Large Language Models: A Survey*, 2024. arXiv: [2312.10997 \[cs.CL\]](#).
- [9] R. Nogueira y K. Cho, *Passage Re-ranking with BERT*, 2020. arXiv: [1901.04085 \[cs.IR\]](#).
- [10] D. Edge et al., *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, 2025. arXiv: [2404.16130 \[cs.CL\]](#).
- [11] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey e I. Sutskever, «Robust Speech Recognition via Large-Scale Weak Supervision,» *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [12] A. van den Oord et al., «WaveNet: A Generative Model for Raw Audio,» en *Proc. 9th ISCA Speech Synthesis Workshop*, 2016.
- [13] J. Kim, J. Kong y J. Son, «Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech,» en *Proceedings of the 38th International Conference on Machine Learning*, 2021.
- [14] L. Zheng et al., *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*, 2023. arXiv: [2306.05685 \[cs.CL\]](#).
- [15] R. Cohen, M. Hamri, M. Geva y A. Globerson, «LM vs LM: Detecting Factual Errors via Cross Examination,» en *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore: Association for Computational Linguistics, 2023, págs. 12 621-12 640.
- [16] A. Hogan et al., «Knowledge Graphs,» *ACM Computing Surveys*, vol. 54, n.º 4, págs. 1-37, 2021.

- [17] A. Louis y A. Nenkova, «Automatic Identification of General and Specific Sentences by Leveraging Discourse Annotations,» en *Proceedings of the 5th International Joint Conference on Natural Language Processing*, 2011, págs. 605-613.
- [18] R. Farkas, V. Vincze, G. Móra, J. Csirik y G. Szarvas, «The CoNLL-2010 Shared Task: Learning to Detect Hedges and their Scope in Natural Language Text,» en *Proceedings of the Fourteenth Conference on Computational Natural Language Learning: Shared Task*, Association for Computational Linguistics, 2010, págs. 1-12.
- [19] G. V. Cormack, C. L. A. Clarke y S. Buettcher, «Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,» en *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ACM, 2009, págs. 758-759.
- [20] D. Goldberg, «What Every Computer Scientist Should Know About Floating-Point Arithmetic,» *ACM Computing Surveys*, vol. 23, n.º 1, págs. 5-48, 1991. DOI: [10.1145/103162.103163](https://doi.org/10.1145/103162.103163)